Illustrative examples of consuming Y2 Web Services

cegid

Illustrative examples of consuming Y2 Web Services

Contents

1. CHANGES IN THE DOCUMENT 4
2. INTRODUCTION 5
3. PHP 6
   - **Installing the environment** 6
     - WSDL Cache 6
   - **Basic authentication** 7
     - Code 7
     - Execution of the script 7
     - Response 7
   - **Complex case (AddNewCustomer)** 7
     - Arrays 8
     - Dates 8
     - Enum 8
     - Non ASCII characters 8
   - **Troubleshooting** 8
4. C# 10
   - **Installing the environment** 10
     - Prerequisites 10
     - Create a new project 10
   - **Import a file** 11
   - **Basic authentication** 12
     - Code 12
     - Running the program 13
     - Response 13
   - **Complex case (AddNewCustomer)** 13
     - Arrays 13
     - Dates 13
     - Enum 13
     - Non ASCII characters 13
   - **Troubleshooting** 13
5. PYTHON 16
   - **Installing the environment** 16
   - **Install a package** 16

Basic authentication **17**
Code 17
Execution of the script 18
Response 18
Complex case (AddNewCutomer) **18**
Arrays 18
Dates 19
Enum 19
Non ASCII characters 19
Troubleshooting **19**
6. RUBY **20**
Installing the environment **20**
Basic authentication **21**
Code 21
Execution of the script 21
Response 21
Note 22
Complex case (AddNewCustomer) **22**
Arrays 22
Dates 22
Enum 22
Non ASCII characters 22
Troubleshooting **22**
7. JAVA **24**
Installing the environment **24**
Prerequisites 24
Install Apache 24
Install Axis2 25
Create a new project 26
Create a Web Service client 28
Import a file **30**
Basic authentication **31**
Code 31
Running the program 32
Response 32
Complex case (AddNewCustomer) **32**
Arrays 32
Dates 32
Enum 32
Non ASCII characters 32
Troubleshooting **33**

8. JAVASCRIPT 36
Installing the environment 36
Disclaimer 36
Basic authentication 37
Code 37
Execution of the script 38
Response 38
Complex case (AddNewCustomer) 38
Arrays 38
Dates 38
Enum 39
Non ASCII characters 39
Troubleshooting 39
9. AUTHENTICATION COOKIE 40
Introduction 40
Sample of a PHP script 40
CookieSoap 40
Code 40
10. APPENDIX 42
basic_auth.php 42
AddNewCustomer.php 42
soapDebug.php 44
test_logs.php 44
basic_auth.cs 44
AddNewCustomer.cs 45
basic_auth.py 47
AddNewCustomer.py 47
test_logs.py 49
basic_auth.rb 49
AddNewCustomer.rb 50
test_logs.rb 51
basic_auth.java 51
AddNewCustomer.java 52
soapclient.js 53
objtree.js 60
CookieSoap.php 68
HelloWorld_cookie.php 68

1. Changes in the document

Date	Author	Type of change
12/24/2014	ABE	Creation
1/19/2015	YPE	Summary review
2/13/2017	TSE	PHP environment review : Add prerequisite PHP version minimum PHP NTLM authentication : Update files to remove hacks
2/16/2017	TSE	PHP environment review : Add wsdl cache configuration for production and test mode
4/12/2017	TSE	Add JavaScript samples
11/16/2020	YPE	To prevent viral detection, sample of code are moved from OLE Object to Appendix Remove NTLM authentication

2. Introduction

In the sample programs of this documentation, you will find:

```
DatabaseId = "DATABASE"
```

"DATABASE" represents the FolderId associated with the database to use when consuming the Web Service; this identifier will be sent in the SOAP request.

Parameters of the HelloWorld method

parameters type *HelloWorld*
HelloWorld method to validate the access to the Web Service from any very light HTTP client via Javascript for example

- text type *string*
  Text to pass to test the consumption.
- clientContext type *RetailContext*
  - DatabaseId type *string*

These programs are provided as examples. In production, the code may be different. Cegid may not be liable for their use.

3. PHP

Installing the environment

- Download PHP for Windows and copy it to C:\php\
  - Note : requires a PHP version >= 5.6.25
- Configure php.ini:
  - Copy C:\php\php.ini-production to C:\php\php.ini
  - Uncomment these lines in php.ini:

Lines	Why
`; extension_dir = "ext"`	Enables the loading of extensions
`; extension=php_openssl.dll`	Enables the SSL support
`; extension=php_soap.dll`	Enables SOAP support (cf. SoapClient class)

WSDL Cache

In production mode

- Change value of the line in php.ini:

Lines	Why
`soap.wsdl_cache_enabled=1`	Enable WSDL cache

Test and debug mode

- Change value of the line in php.ini:

Lines	Why
`soap.wsdl_cache_enabled=0`	Disable WSDL cache

Important: In **production mode**, make sure to set soap.wsdl_cache_enabled=1 to enable cache.

Basic authentication

Code

Script used to consume the Web Service with a Basic authentication (CBR).

basic_auth.php

Sample of code and description:

Code	Info
`<?php` `  $url = "http://localhost/CBR_12.1/WorkerProcessService.svc";`	URL of the Web Service

`  $client = new SoapClient( $url . "?singleWsdl",`
`  array(`
`  "location" => $url,`
`  "login" => "DOMAIN\\USERNAME",`
`  "password" => "PASSWORD"`
`  )`
`  );`
	Create the client
	Link to the WSDL file
	
	Database name, identifier and password

`  $request = new StdClass();`
`  $request->text = "ttt";`
`  $request->clientContext = new StdClass();`
`  $request->clientContext->DatabaseId = "DATABASE";`
	Prepare the request

`  $resu = $client->HelloWorld($request);`
`  print_r( $resu );`
`?>`
	Call the HelloWorld method
	Display the result

The created request corresponds to the following XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ns1="http://www.cegid.fr/Retail/1.0">
  <SOAP-ENV:Body>
    <ns1:HelloWorld>
      <ns1:text>ttt</ns1:text>
      <ns1:clientContext>
        <ns1:DatabaseId>DATABASE</ns1:DatabaseId>
      </ns1:clientContext>
    <ns1:HelloWorld>
  <SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Execution of the script

Open a command prompt, access the directory storing the script and enter this line:

```
C:\php\php.exe -f basic_auth.php
```

Response

The program displays the response from the Web Service, specifying the type of authentication used: Basic (CBR)

```
stdClass Object
(
    [HelloWorldResult] => Now: (2014-11-12T11:26:43.2069451)
ApplicationBase: (C:\inetpub\wwwroot\Cegid Retail\CBR 12.1\)
InputText: (ttt)
DataBaseId: (DATABASE)
ErpIdentity: (CEG) (CEGID) (DOMAIN)
Current Identity: (DOMAIN\CEGID) (CBR)
CurrentCulture: (fr-FR)
CurrentUICulture: (fr-FR)
CBR Version: (12.1.0.1057)
)
```

Complex case (AddNewCustomer)

How can you create more complex types? See example with AddNewCustomer:

AddNewCustomer.php

Arrays

Create each cell and create an array with `array();`

```
$UserDefinedBoolean1 = new StdClass();
$UserDefinedBoolean1->Id = 1;
$UserDefinedBoolean1->Value = true;
$UserDefinedBoolean2 = new StdClass();
$UserDefinedBoolean2->Id = 2;
$UserDefinedBoolean2->Value = false;
$UserDefinedBoolean3 = new StdClass();
$UserDefinedBoolean3->Id = 3;
$UserDefinedBoolean3->Value = true;

$request->customerData->UserDefinedBooleans = array($UserDefinedBoolean1, $UserDefinedBoolean2,
$UserDefinedBoolean3);
```

Dates

Create a DateTime object, specify the date and time, then convert the object into a character string with `$date->format`

```
$UserDefinedDate1 = new StdClass();
$UserDefinedDate1->Id = 1;

$date = new Datetime(null, new DateTimeZone("UTC"));
$date->SetDate(1970, 01, 01);
$date->SetTime(18, 00, 00);

$UserDefinedDate1->Value = $date->format("Y-m-d\TH:i:s");
```

Enum

There is no Enum in PHP, so go directly to the constant value

```
$request->customerData->AddressData->CountryIdType = "Internal";
```

Non ASCII characters

Do not forget to save the script with UTF-8 encoding.

```
$UserDefinedText2->Value = "漢字";
```

Troubleshooting

Traces enable the recovery of the XML input in order to process the case in SoapUI again, for example, and the XML output to get the entire error message. This paragraph describes how to activate the traces.

To enable the display of the traces, first place the "soapDebug.php" file in the same directory as the script.

soapDebug.php

Then include "soapDebug.php" by adding the following line to the beginning of the script.

```
include('soapDebug.php');
```

And add this line after having called the Web Service:

```
soapDebug($client);
```

where `$client` is an instance of SoapClient

Do not forget to catch the exception that may be launched by `$client`, because if an error occurs the program will abort and the traces will not be displayed.

When the script is run, you will get the XML request as well as the XML response.

Script test_logs.php and example of traces:

log.txt

4. C#

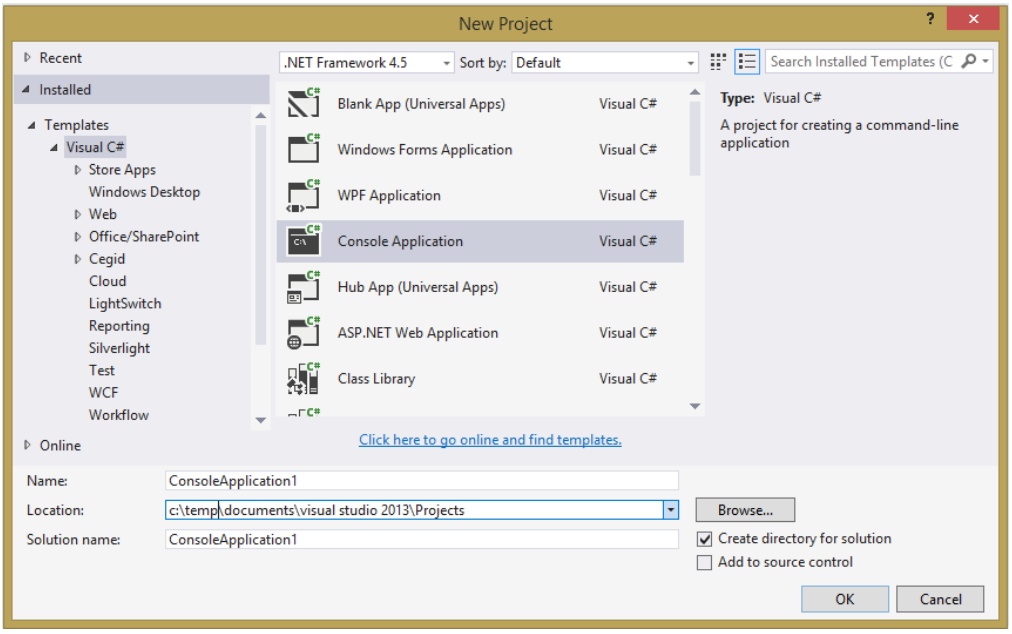
Installing the environment

Prerequisites

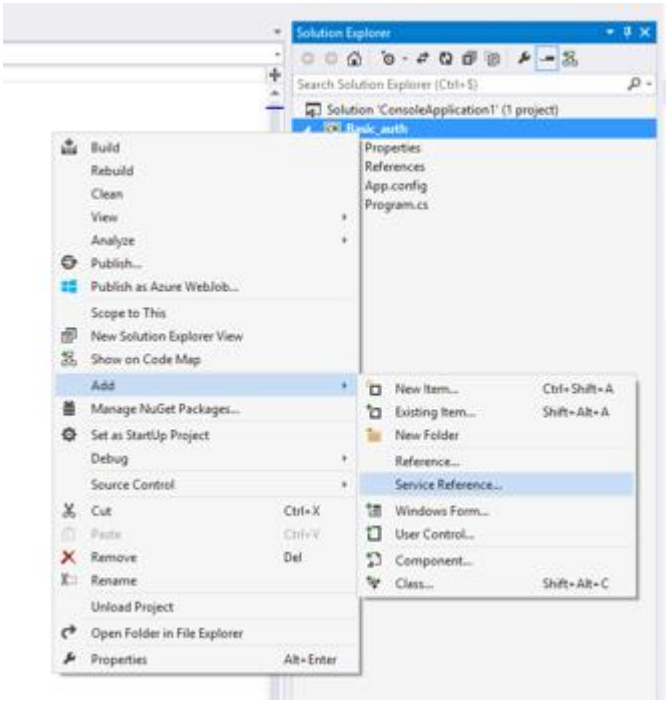
[Download and install Visual Studio]

Create a new project

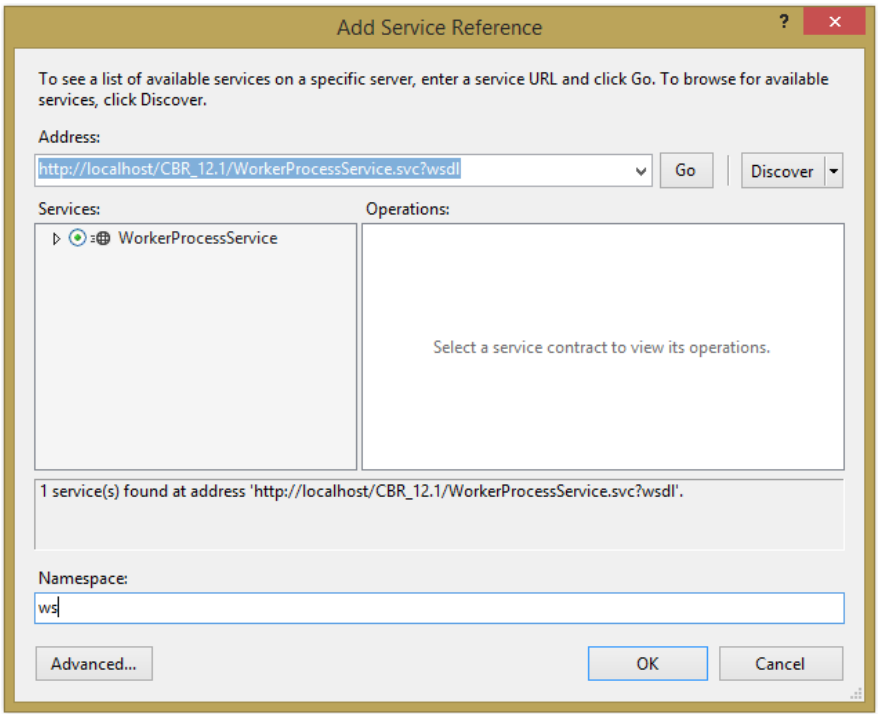
To create a project, go to "**File** / **New** / **Project**".



Select "**Console Application**" from **Visual C#**; enter the project name and click on **OK**.



In the **Solution Explorer**, right-click on the project name and "**Add Service Reference...**"

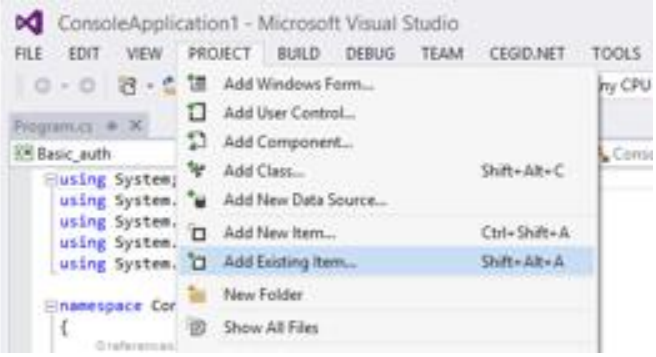


Specify the URL of the WSDL in field "**Address**" and click on "**Go**".

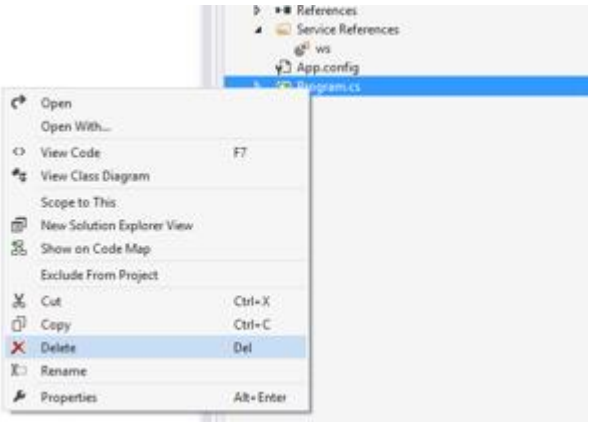
Select the Web Service and specify a name in field "**Namespace**" and click on **OK**.

Import a file

This paragraph explains how to import a file in Visual Studio in order to use the sample files about the consumption of Web Services.



To add a file click on "**Project / Add Existing Item...**"



Please notice:

Visual Studio creates a Program.cs file containing a "main" function.

You must delete the program for the project to compile with the imported file.

Basic authentication

Code

Source code used to consume the Web Service with a Basic authentication (CBR).

basic_auth.cs

Sample of code and description:

Code	Info

`static void Main(string[] args)`
`{`
`  var bhb = new BasicHttpBinding(`
`  BasicHttpSecurityMode.TransportCredentialOnly`
`  );`
	For HTTPS connections:
	Transport
	For HTTP connections:
	TransportCredentialOnly

`  var auth = HttpClientCredentialType.Basic;` `  bhb.Security.Transport.ClientCredentialType = auth;`	BASIC authentication type

`  var service = new WorkerProcessServiceClient(`
`  bhb,`
`  new EndpointAddress(`
`  http://localhost/CBR_12.1/WorkerProcessService.svc`
`  )`
`  );`
	URL of the Web Service

`  service.ClientCredentials.UserName.UserName = "DOMAIN\\USERNAME";` `  service.ClientCredentials.UserName.Password = "PASSWORD";`	Database name, identifier and password

`  var text = "ttt";`
`  var clientContext = new RetailContext();`
`  clientContext.DatabaseId = "DATABASE";`
	Prepare the request

`  var result = service.HelloWorld(text, clientContext);`	Call the HelloWorld method

`  Console.WriteLine(result);`
`  Console.ReadLine();`
`}`
	Display the result

The created request corresponds to the following XML:

```
<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/">
  <s:Body>
    <HelloWorld xmlns="http://www.cegid.fr/Retail/1.0">
      <text>ttt</text>
      <clientContext xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
        <DatabaseId>DATABASE</DatabaseId>
```

```
      </clientContext>
    </HelloWorld>
  </s:Body>
</s:Envelope>
```

Running the program

To launch the program, click on (or press key F5)

Response

The program displays the response from the Web Service, specifying the type of authentication used: Basic (CBR)

```
Now: (2014-11-13T09:40:07.7248033)
ApplicationBase: (C:\inetpub\wwwroot\Cegid Retail\CBR 12.1\)
InputText: (ttt)
DataBaseId: (DATABASE)
ErpIdentity: (CEG) (CEGID) (DOMAIN)
Current Identity: (DOMAIN\CEGID) (CBR)
CurrentCulture: (fr-FR)
CurrentUICulture: (fr-FR)
CBR Version: (12.1.0.1057)
```

Complex case (AddNewCustomer)

How can you to create more complex types? See example with AddNewCustomer:

AddNewCustomer.cs

Arrays

Create a list of several items (in this example UserDefinedBoolean). And call the ToArray() method to create an array from the list.

```
List<UserDefinedBoolean> userDefinedBooleanList = new List<UserDefinedBoolean>()
            {
                new UserDefinedBoolean(){ Id = UserDefinedId._1, Value = true },
                new UserDefinedBoolean(){ Id = UserDefinedId._2, Value = false },
                new UserDefinedBoolean(){ Id = UserDefinedId._3, Value = true }
            };
customerData.UserDefinedBooleans = userDefinedBooleanList.ToArray();
```

Dates

Create a DateTime object by specifying the year, month, day, hour, minutes and seconds.

```
Value = new DateTime(1970, 01, 01, 18, 00, 00)
```

Enum

The proxy generates enum types from the WSDL.

```
customerData.AddressData.CountryIdType = CountryIdType.Internal;
```

Non ASCII characters

Do not forget to save the script with UTF-8 encoding; Visual Studio does it by default.

```
Value = "漢字"
```

Troubleshooting

Traces enable the recovery of the XML input in order to process the case in SoapUI again, for example, and the XML output to get the entire error message. This paragraph describes how to activate the traces.

To enable the display of traces, you must add the following lines in the app.config file of the project.

```
<system.diagnostics>
  <sources>
      <source name="System.ServiceModel.MessageLogging">
        <listeners>
                 <add name="messages"
                 type="System.Diagnostics.XmlWriterTraceListener"
                 initializeData="c:\logs\messages.svclog" />
          </listeners>
      </source>
    </sources>
</system.diagnostics>

<system.serviceModel>
  <diagnostics>
    <messageLogging
         logEntireMessage="true"
         logMalformedMessages="false"
         logMessagesAtServiceLevel="true"
         logMessagesAtTransportLevel="false"
         maxMessagesToLog="5000"
         maxSizeOfMessageToLog="2000000"/>
  </diagnostics>
</system.serviceModel>
```

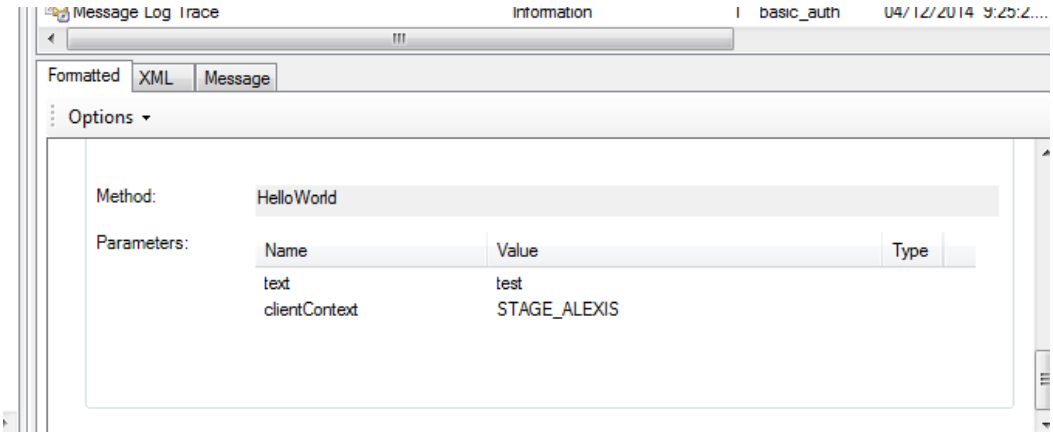
When running the program, file "messages.svclog" containing the XML request and the response is created in C:\logs\

Example of traces:

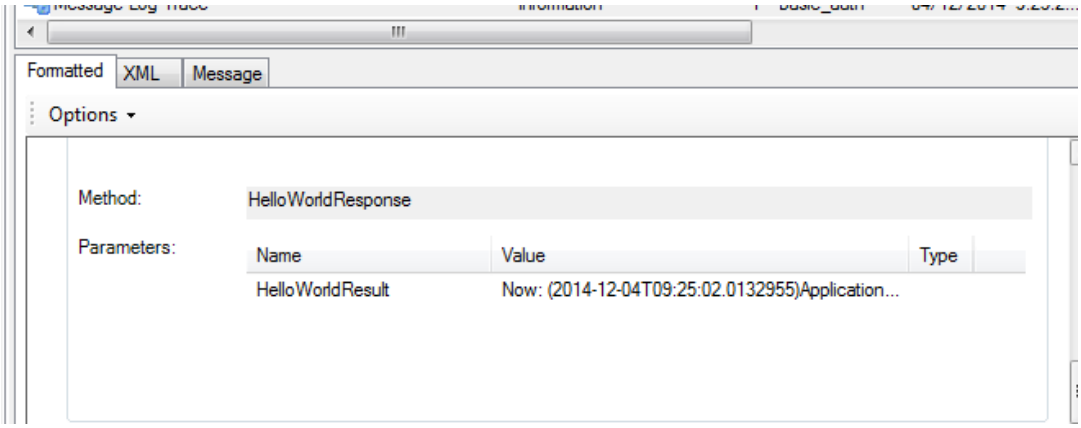
messages.svclog

The ".svclog" extension is associated with an MS tool to view the WCF log: [Microsoft Service Trace Viewer]

Request



Reply


Formatted
XML
Message
Options
Method:
HelloWorldResponse
Parameters:
Name
Value
Type
HelloWorldResult
Now: (2014-12-04T09:25:02.0132955)Application...

5. Python

Installing the environment

Download and install Python 2.7

Install these two packages (see how to install a package):

- setuptools (1.4.2)
- suds (0.4)

With the **suds** proxy, you can create a client with this function:

```
client = suds.client.Client(url, username=username, password=password)
```

And get information about the client by using simply:

```
print client
```

This returns with the **WorkerProcessService** Web Service the following result:

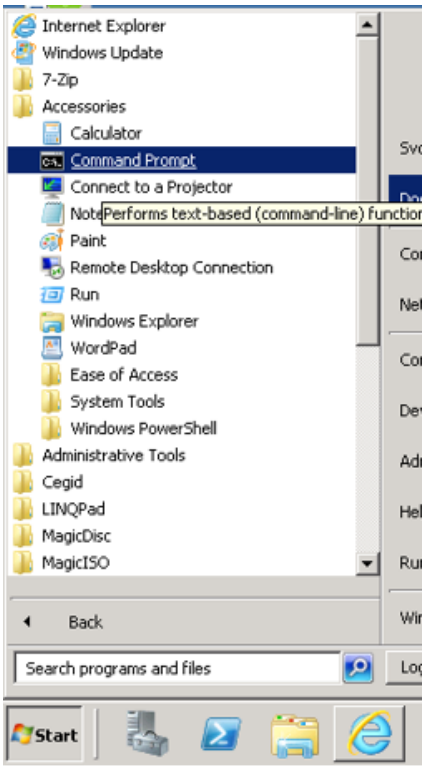
```
Suds ( https://fedorahosted.org/suds/ )  version: 0.4 GA  build: R699-20100913

Service ( WorkerProcessService ) tns="http://tempuri.org/"
   Prefixes (4)
      ns0 = "http://schemas.microsoft.com/2003/10/Serialization/"
      ns1 = "http://schemas.microsoft.com/2003/10/Serialization/Arrays"
      ns2 = "http://www.cegid.fr/Retail/1.0"
      ns3 = "http://www.cegid.fr/fault"
   Ports (1):
      (BasicHttpBinding_IWorkerProcessService)
         Methods (2):
            HelloWorld(xs:string text, ns2:RetailContext clientContext, )
            Ping(xs:string input, ns2:RetailContext clientContext, )
         Types (8):
            ns1:ArrayOfstring
            ns3:BusinessFaultDetail
            ns3:CbpFaultDetail
            ns2:PingReply
            ns2:RetailContext
            ns0:char
            ns0:duration
            ns0:guid
```

Here, the **HelloWorld** method uses as parameter a character string and a **RetailContext** object; to create the latter, you may use:

```
clientContext = client.factory.create('ns2:RetailContext')
```

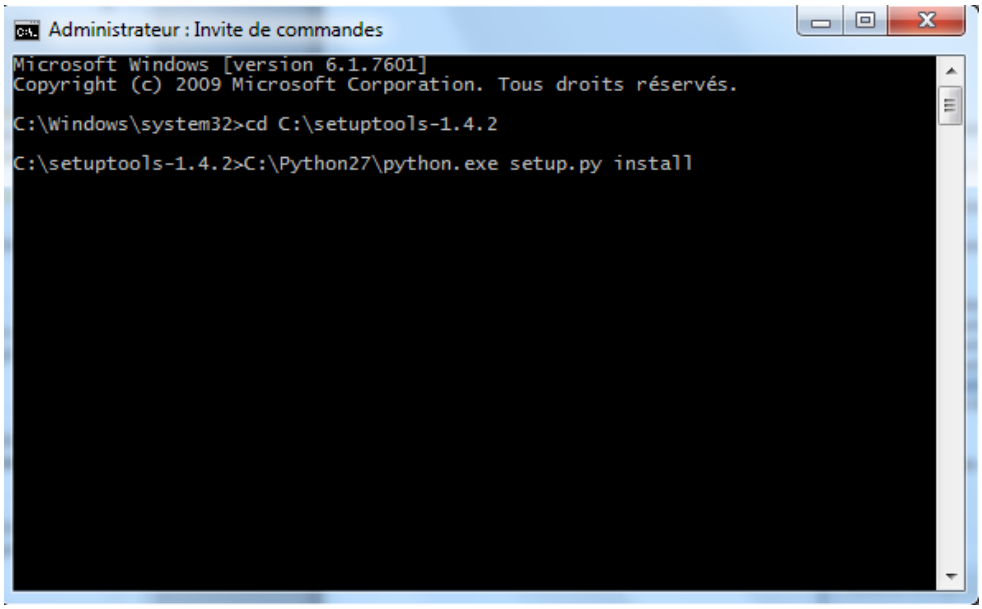
Install a package

Extract the package (in the example, **setuptools** is extracted to C:\)



From menu **Start**, go to **Accessories**

Then right-click on **Command prompt** and select **Run as administrator**.



Once the command prompt window displays, position to the directory of the package and run the following command:

C:\Python27\python.exe setup.py install

Basic authentication

Code

Script used to consume the Web Service with a Basic authentication (CBR).

basic_auth.py

Sample of code and description:

Code	Info
`import suds`	Include the suds library
`url = 'http://localhost/CBR_12.1/WorkerProcessService.svc?wsdl'`	Link to the WSDL file
`username = "DOMAIN\\USER"` `password = "PASSWORD"`	Database name, identifier and password
`client = suds.client.Client(url, username=username, password=password)`	Create the client
`clientContext = client.factory.create('ns2:RetailContext')`	Prepare the request
`text = "test"` `clientContext.DatabaseId = "DATABASE"`	Call the HelloWorld method
`response = client.service.HelloWorld(text, clientContext)`	Display the result
`print response`	

The created request corresponds to the following XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<SOAP-ENV:Envelope xmlns:ns0=http://www.cegid.fr/Retail/1.0
xmlns:ns1=http://schemas.xmlsoap.org/soap/envelope/
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <ns1:Body>
    <ns0:HelloWorld>
      <ns0:text>test</ns0:text>
      <ns0:clientContext>
        <ns0:DatabaseId>DATABASE</ns0:DatabaseId>
      </ns0:clientContext>
    </ns0:HelloWorld>
  </ns1:Body>
</SOAP-ENV:Envelope>
```

Execution of the script

Open a command prompt, access the directory storing the script and enter this line:

```
C:\Python27\python.exe basic_auth.py
```

Response

The program displays the response from the Web Service, specifying the type of authentication used: Basic (CBR)

```
Now: (2014-11-14T13:36:33.8244203)
ApplicationBase: (C:\inetpub\wwwroot\Cegid Retail\CBR 12.1\)
InputText: (test)
DataBaseId: (DATABASE)
ErpIdentity: (CEG) (CEGID) (DOMAIN)
Current Identity: (DOMAIN\CEGID) (CBR)
CurrentCulture: (fr-FR)
CurrentUICulture: (fr-FR)
CBR Version: (12.1.0.1057)
```

Complex case (AddNewCutomer)

How can you create more complex types? See example with AddNewCustomer:

AddNewCustomer.py

Arrays

Create each of the cells and add them to the array

```
UserDefinedBoolean1 = client.factory.create('ns2:UserDefinedBoolean')
UserDefinedBoolean1.Id = 1
UserDefinedBoolean1.Value = True
UserDefinedBoolean2 = client.factory.create('ns2:UserDefinedBoolean')
UserDefinedBoolean2.Id = 2
UserDefinedBoolean2.Value = False
UserDefinedBoolean3 = client.factory.create('ns2:UserDefinedBoolean')
```

```
UserDefinedBoolean3.Id = 3
UserDefinedBoolean3.Value = True

customerData.UserDefinedBooleans.UserDefinedBoolean.append(UserDefinedBoolean1)
customerData.UserDefinedBooleans.UserDefinedBoolean.append(UserDefinedBoolean2)
customerData.UserDefinedBooleans.UserDefinedBoolean.append(UserDefinedBoolean3)
```

Dates

Import the DateTime library and specify a date with the datetime method.

```
UserDefinedDate1.Value = datetime.datetime(1970, 01, 01, 18, 00, 00)
```

Enum

There is no Enum in Python, so go directly to the constant value

```
customerData.AddressData.CountryIdType = "Internal"
```

Non ASCII characters

Do not forget to save the script with UTF-8 encoding. Character strings with non ASCII characters must be declared with UTF-8 encoding and the response of the program must be encoded in UTF-8 too.

```
UserDefinedText2.Value = unicode("漢字", "utf-8")

print response.encode("utf-8")
```

Troubleshooting

Traces enable the recovery of the XML input in order to process the case in SoapUI again, for example, and the XML output to get the entire error message. This paragraph describes how to activate the traces.

Therefore, you must first import the "logging" library into the script:

```
import logging
```

And add these two lines:

```
logging.basicConfig(level=logging.INFO)
logging.getLogger('suds.client').setLevel(logging.DEBUG)
```

When the script is run, you will get the XML request as well as the XML response.

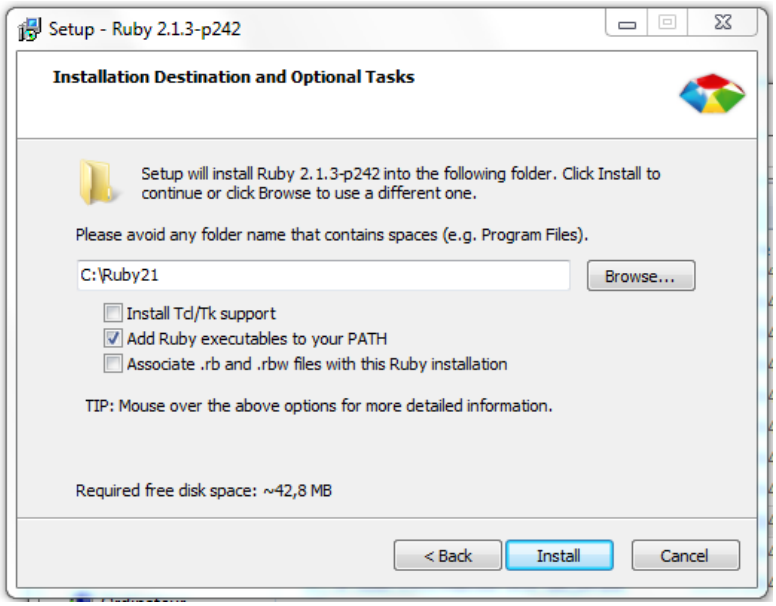
Script test_logs.py and example of traces:

log.txt

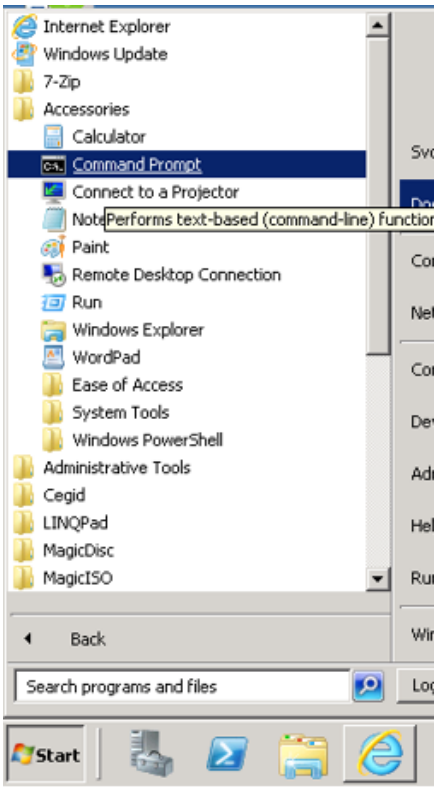
6. Ruby

Installing the environment

Download and install Ruby



Do not forget to tick the checkbox "**Add Ruby executables to your PATH**"



From menu **Start**, go to **Accessories**

Then right-click on **Command prompt** and select **Run as administrator**.

Enter the following command:

```
gem install savon
```

Basic authentication

Code

Script used to consume the Web Service with a Basic authentication (CBR).

basic_auth.rb

Sample of code and description:

Code	Info
`require 'savon'`	Include the savon library
`client = Savon.client do` `  wsdl "http://localhost/CBR_12.1/WorkerProcessService.svc?wsdl"`	Create the client Link to the WSDL file

`  basic_auth(`
`  "DOMAIN\\USERNAME",`
`  "PASSWORD"`
`  )`
	Database name, identifier and password

`  namespace "http://www.cegid.fr/Retail/1.0"` `end`	Namespace of the Web Service

`message ={`
`  "wsdl:text"=>"test",`
`  "wsdl:clientContext"=>{`
`  "wsdl:DatabaseId"=>"DATABASE"`
`  }`
`}`
	Prepare the request

`response = client.call(:hello_world, message: message)`	Call the HelloWorld method
`puts response.to_xml.gsub("
", "")`	Display the result

The created request corresponds to the following XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<env:Envelope xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:wsdl="http://www.cegid.fr/Retail/1.0"
xmlns:env="http://schemas.xmlsoap.org/soap/envelope/">
  <env:Body>
    <wsdl:HelloWorld>
      <wsdl:text>test</wsdl:text>
      <wsdl:clientContext>
        <wsdl:DatabaseId>DATABASE</wsdl:DatabaseId>
      </wsdl:clientContext>
    </wsdl:HelloWorld>
  </env:Body>
</env:Envelope>
```

Execution of the script

Open a command prompt, access the directory storing the script and enter this line:

```
ruby basic_auth.rb
```

Response

The program displays the response from the Web Service, specifying the type of authentication used: Basic (CBR)

```
<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/"><s:Body><HelloWorldResponse
xmlns="http://www.cegid.fr/Retail/1.0"><HelloWorldResult>Now: (2014-11-17T11:10:34.6394072)
ApplicationBase: (C:\inetpub\wwwroot\Cegid Retail\CBR 12.1\)
InputText: (test)
DataBaseId: (DATABASE)
ErpIdentity: (CEG) (CEGID) (DOMAIN)
Current Identity: (DOMAIN\CEGID) (CBR)
CurrentCulture: (fr-FR)
CurrentUICulture: (fr-FR)
CBR Version: (12.1.0.1057)</HelloWorldResult></HelloWorldResponse></s:Body></s:Envelope>
```

Note

The various fields of the XML message are in an array of the response sent by client.call

For example, to display the character string returned by HelloWorld, you will use:

```
puts response.body[:hello_world_response][:hello_world_result]
```

You can display the whole content of the array with:

```
puts response.body
```

Complex case (AddNewCustomer)

How can you create more complex types? See example with AddNewCustomer:

AddNewCustomer.rb

Arrays

Create an array and populate each cell.

```
"wsdl:UserDefinedBoolean"=>[
        {
                "wsdl:Id"=>1,
                "wsdl:Value"=>true
        },
        {
                "wsdl:Id"=>2,
                "wsdl:Value"=>false
        },
        {
                "wsdl:Id"=>3,
                "wsdl:Value"=>true
        }
        ]
```

Dates

Create a DateTime object, specify the date and time and convert the object into a character string by the means of the `strftime()` method.

```
"wsdl:Value"=>DateTime.new(1970, 01, 01, 18, 00, 00).strftime("%Y-%m-%dT%H:%M:%S")
```

Enum

There is no Enum in Ruby, so go directly to the constant value

```
"wsdl:CountryIdType"=>"Internal"
```

Non ASCII characters

Do not forget to save the script with UTF-8 encoding.

```
"wsdl:Value"=>"漢字"
```

Troubleshooting

Traces enable the recovery of the XML input in order to process the case in SoapUI again, for example, and the XML output to get the entire error message. This paragraph describes how to activate the traces.

To enable the display of the traces, the following two parameters must be added when creating the client.

```
-    log true
-    pretty_print_xml true
```

This will result in:

```
client = Savon.client do
      wsdl "http://localhost/CBR_12.1/WorkerProcessService.svc?wsdl"
      basic_auth("DOMAIN\\USERNAME", "PASSWORD")
      namespace "http://www.cegid.fr/Retail/1.0"
      log true
      pretty_print_xml true
end
```

When the script is run, you will get the XML request as well as the XML response.

Script test_logs.rb and example of traces:

log.txt

7. Java

Installing the environment

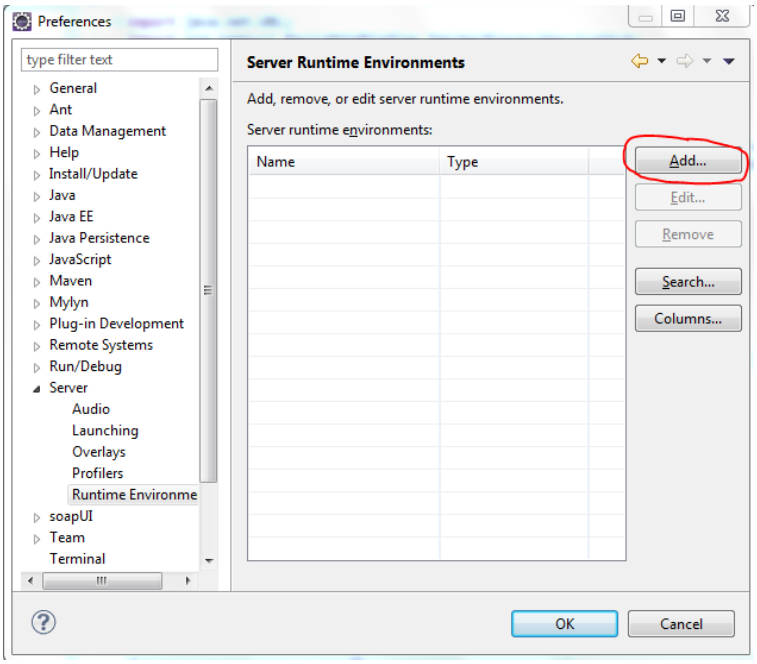
Prerequisites

- Download and install Eclipse (Kepler)
- Download Apache Tomcat v7.0.56 and extract the content from the archive
- Download Axis 2 and extract the content from the archive

Install Apache

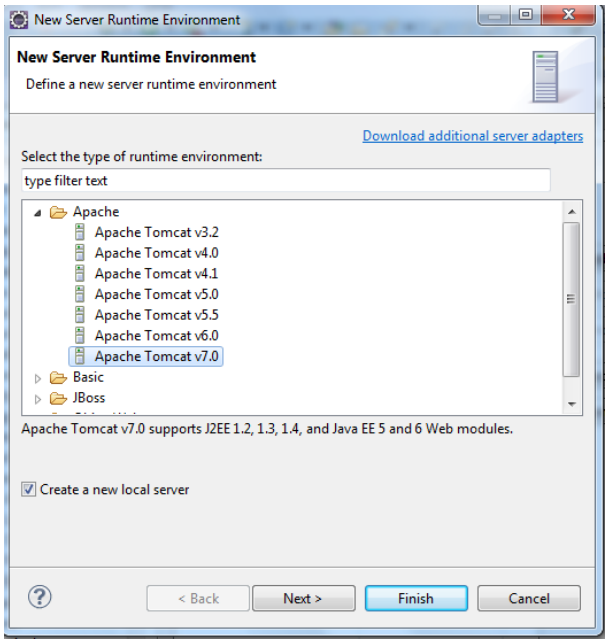
Apache is a HTTP server required for the use of Axis2

Open **Eclipse**

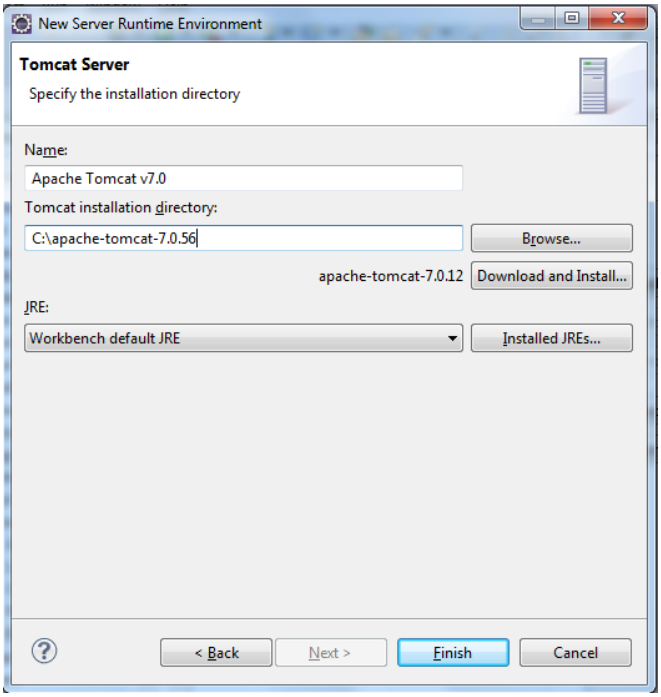


Go to **Windows/Preferences**

In the menu on the left, select **Server/Runtime Environment** and click on **Add**.



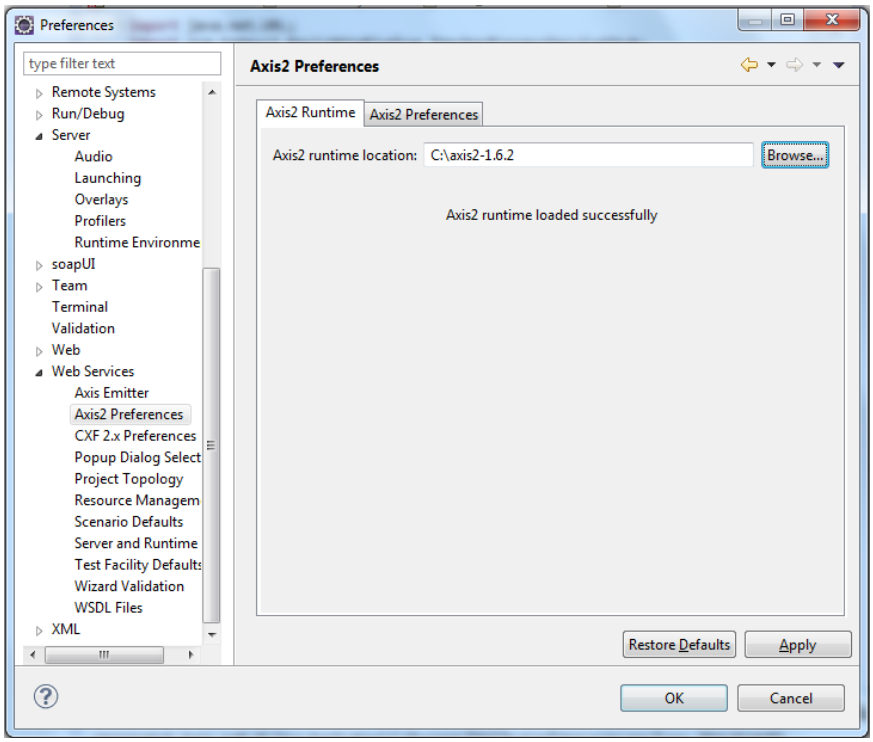
Select **Apache Tomcat v7.0** and **Next**



Click on **Browse** and specify the installation directory for **Apache Tomcat** (the folder extracted from the archive) and click on **Finish**

Install Axis2

Axis2 is an API used to develop/consume SOAP Web Services.

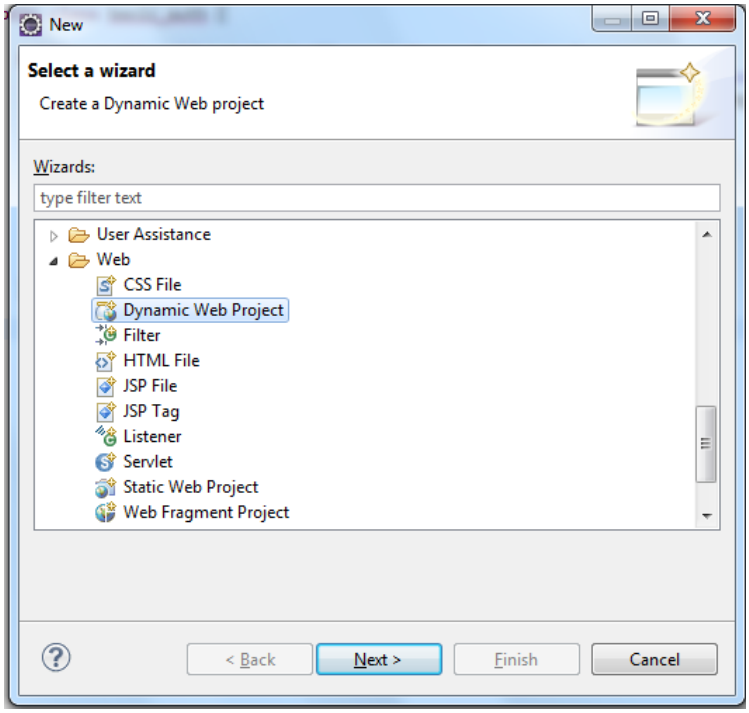


Once again go to **Windows**/**Preferences**, select **Web Services**/**Axis2 Preferences**.

Click on **Browse** and specify the installation directory for **Axis2** (the folder extracted from the archive)

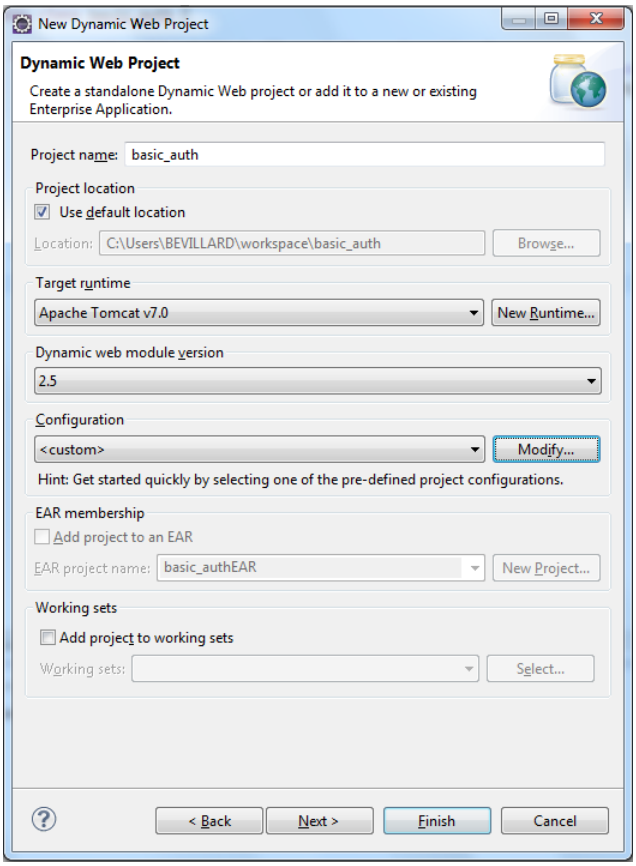
Then, click on **OK**

Create a new project



Go to **File**/**New**/**Other...**

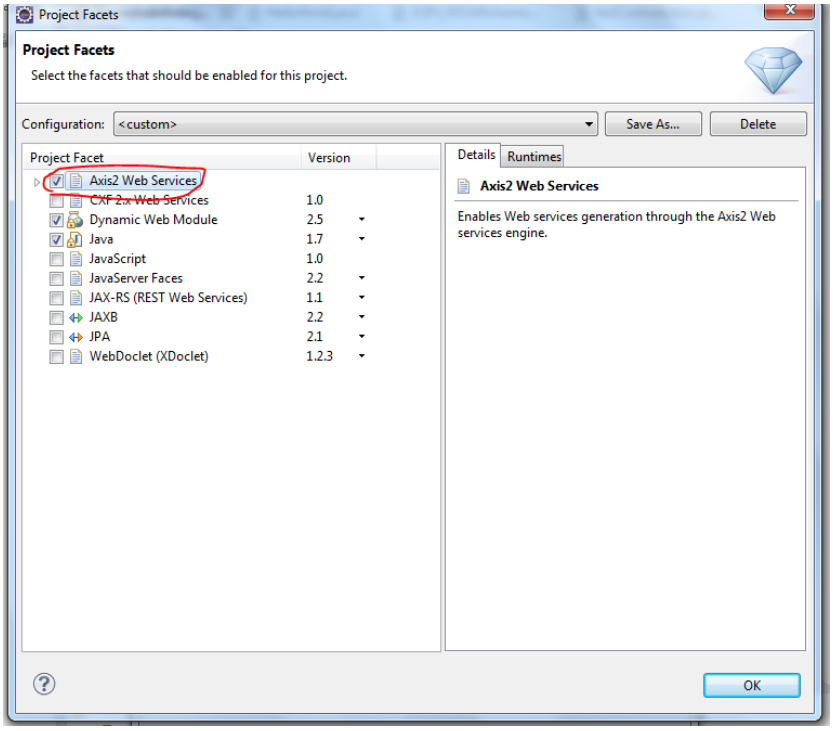
Under **Web**, click on **Dynamic Web Project** and on **Next**



Specify the name of the project

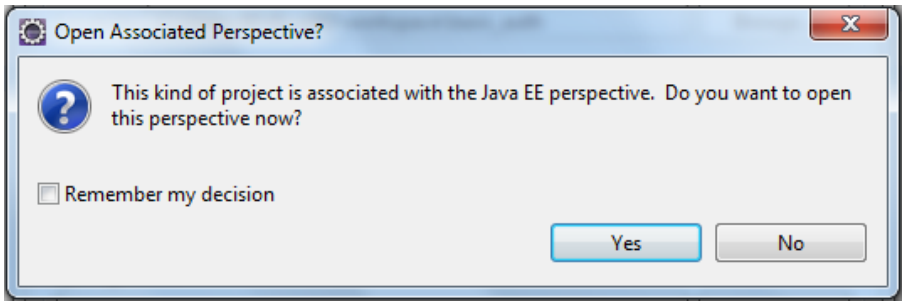
Select **Apache Tomcat v7.0** for **Target runtime**, and **2.5** for **Dynamic web module version**

Click on **Modify…**



Tick option **Axis2 Web Services** and click on **OK**

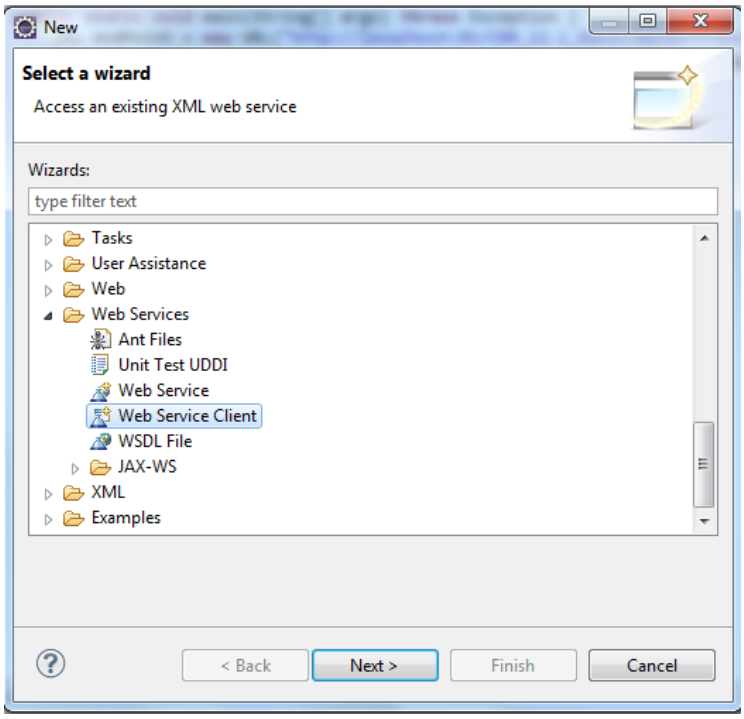
Then click on **Finish**



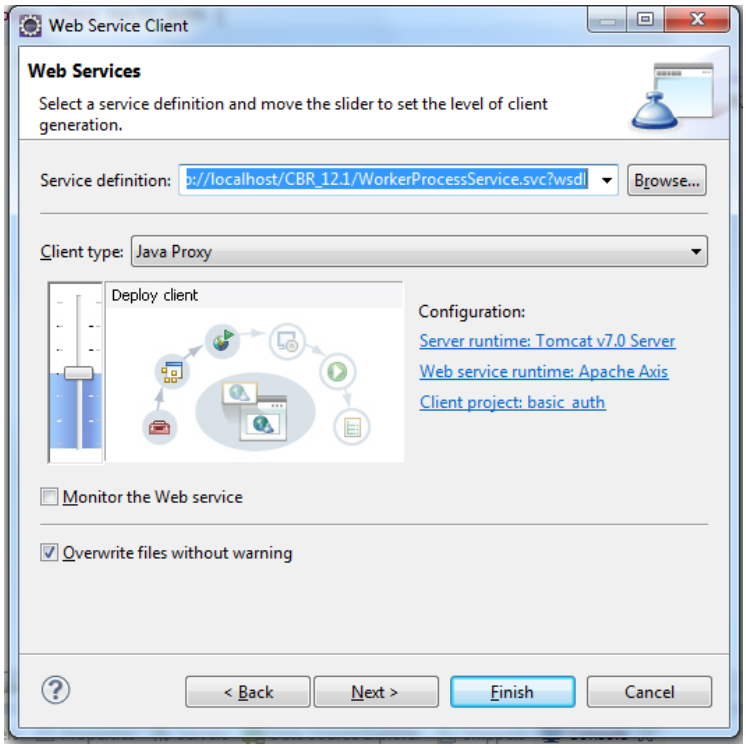
If Eclipse prompts you to open the **Java EE** mode, click on **Yes**

Create a Web Service client

Right-click on the project in the **Project Explorer** and click on **New**/**Other...**

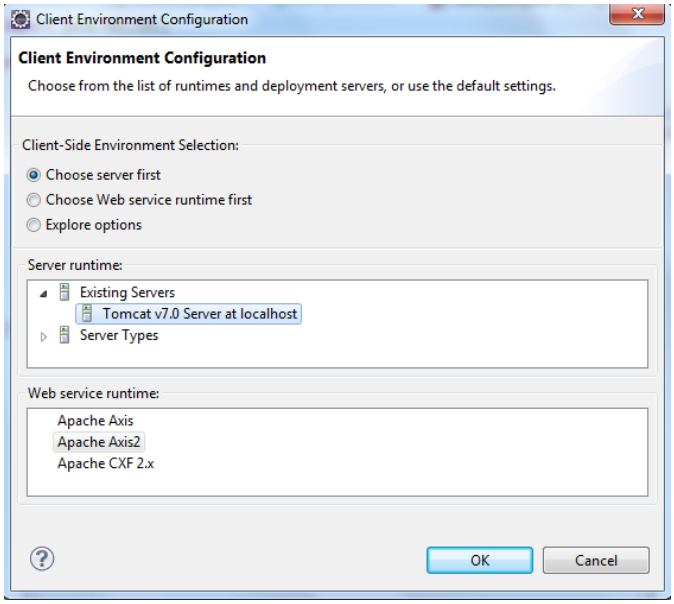


Select **Web Service Client** and click on **Next**

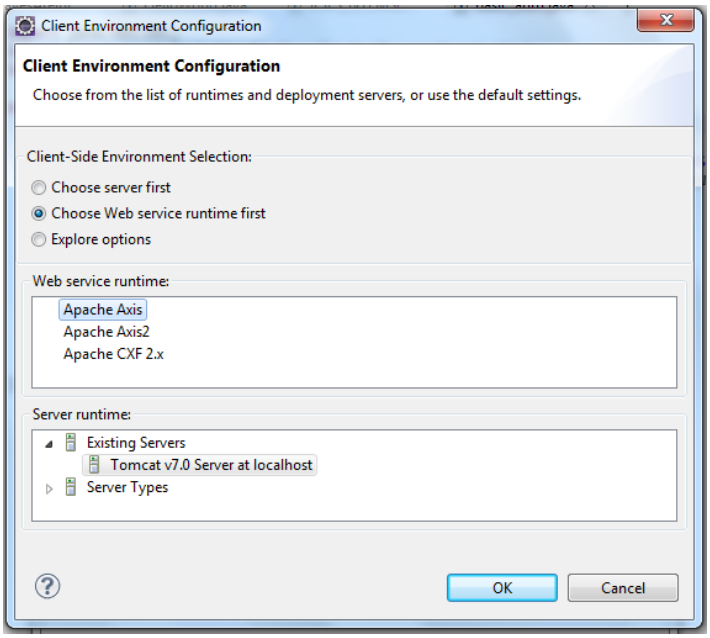


Specify the link to the WSDL file in **Service definition**

Check that under **Configuration**, you will find **Tomcat Server v7.0** as **Server runtime** and **Apache Axis** as **Web service runtime**. Otherwise, click on one of them, and select the right configuration:



Server runtime

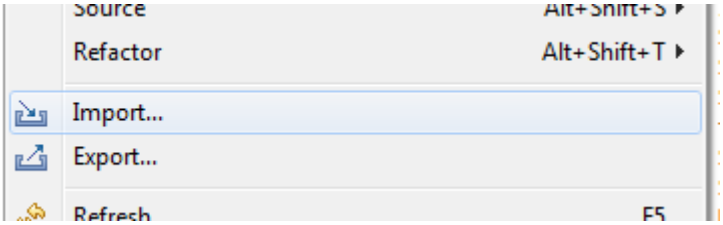


Web service runtime

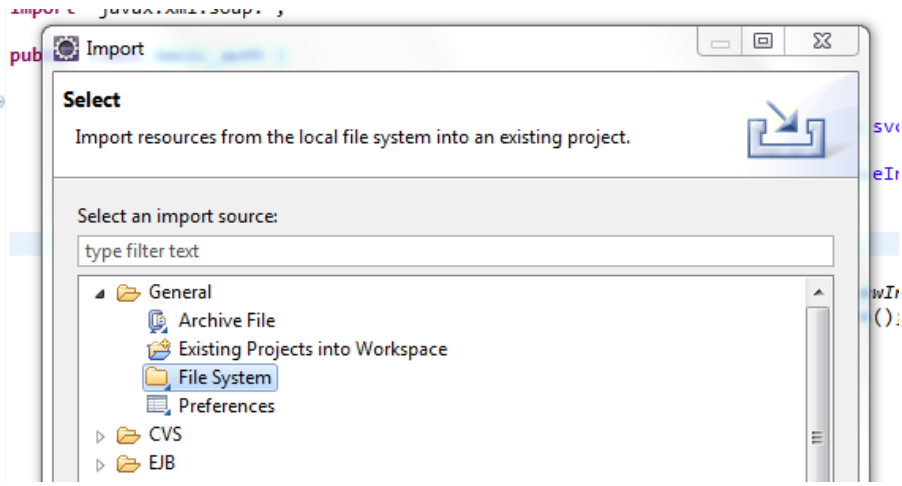
Click on **Finish**

Import a file

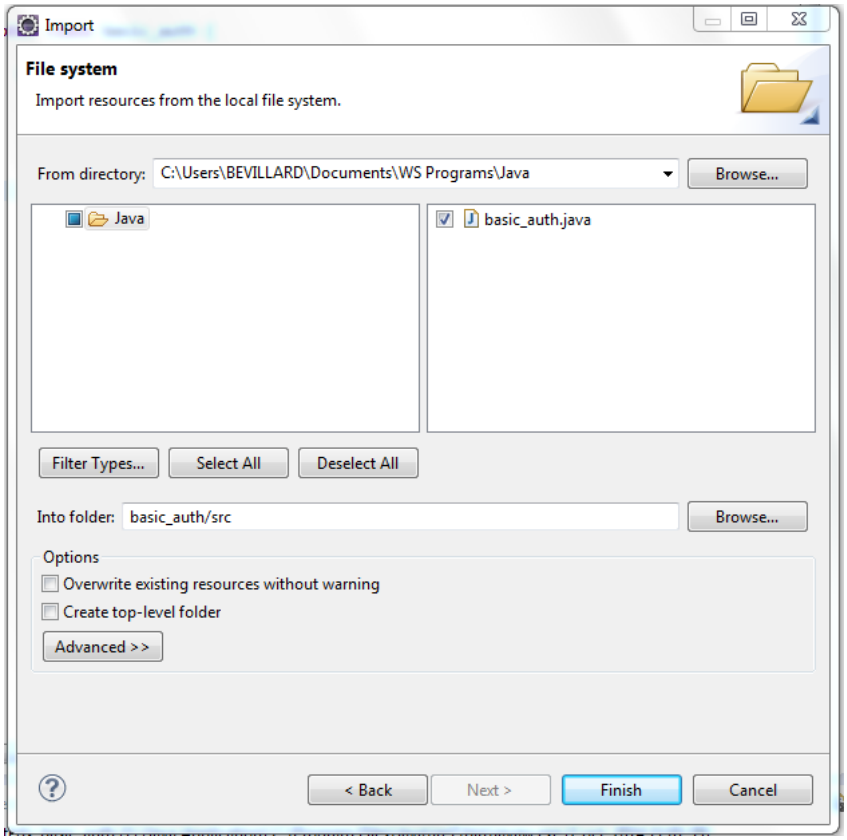
This paragraph explains how to import a file in Eclipse in order to use the sample files about the consumption of Web Services.



In the **Package Explorer**, develop the project as well as **Java Resources**, right-click on **src** and click on **Import...**



Under **General**, select **File System** and click on **Next**



Click on **Browse...** and select the folder containing the file to import

The content of the directory will appear, tick the checkbox next to the file to import and click on "**Finish**"

Basic authentication

Code

Source code used to consume the Web Service with a Basic authentication (CBR).

basic_auth.java

Sample of code and description:

Code	Info
`public static void main(String[] args) throws Exception {` `  WorkerProcessServiceLocator ws = new WorkerProcessServiceLocator();`	Create the client

`  BasicHttpBinding_IWorkerProcessServiceStub stub = (BasicHttpBinding_IWorkerProcessServiceStub)ws.getBasicHttpBinding_IWorkerProcessService();`
`  stub.setUsername("DOMAIN\\USERNAME");`
`  stub.setPassword("PASSWORD");`
	Database name, identifier and password

`  String text = "test";`
`  RetailContext clientContext = new RetailContext();`
`  clientContext.setDatabaseId("DATABASE");`
	Prepare the request

`  String response = stub.helloWorld(text, clientContext);`	Call the HelloWorld method
`  System.out.println(response);` `}`	Display the result

The created request corresponds to the following XML:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance">
  <soapenv:Body>
    <HelloWorld xmlns="http://www.cegid.fr/Retail/1.0">
      <text>test</text>
      <clientContext>
        <DatabaseId>DATABASE</DatabaseId>
      </clientContext>
    </HelloWorld>
  </soapenv:Body>
</soapenv:Envelope>
```

Running the program

To launch the program, click on (or press key F11)

If Eclipse prompts you to launch the program, select **Java Application**

Response

The program displays the response from the Web Service, specifying the type of authentication used: Basic (CBR)

```
Now: (2014-11-17T16:11:24.9651397)
ApplicationBase: (C:\inetpub\wwwroot\Cegid Retail\CBR 12.1\)
InputText: (test)
DataBaseId: (DATABASE)
ErpIdentity: (CEG) (CEGID) (DOMAIN)
Current Identity: (DOMAIN\CEGID) (CBR)
CurrentCulture: (fr-FR)
CurrentUICulture: (fr-FR)
CBR Version: (12.1.0.1057)
```

Complex case (AddNewCustomer)

How can you create more complex types? See example with AddNewCustomer:

AddNewCustomer.java

Arrays

Create an array and populate each cell.

```
UserDefinedBoolean[] userDefinedBooleans = new UserDefinedBoolean[] {
                new UserDefinedBoolean(UserDefinedId.value1, true),
                new UserDefinedBoolean(UserDefinedId.value2, false),
                new UserDefinedBoolean(UserDefinedId.value3, true)
};
customerData.setUserDefinedBooleans(userDefinedBooleans);
```

Dates

Create a `Calendar` object by the means of `Calendar.getInstance()` and specify the date and time by the means of the `set();` method.

Please notice, the month is an enum and starts with 0 (January = 0, February = 1 ... December = 11)

```
Calendar date = Calendar.getInstance();
date.set(1970, Calendar.JANUARY, 01, 18, 00, 00);
```

Enum

The proxy generates enum types from the WSDL.

```
addressData.setCountryIdType(CountryIdType.Internal);
```

Non ASCII characters

Do not forget to save the script with UTF-8 encoding; Eclipse does it by default.

```
new UserDefinedText(UserDefinedId.value2, "漢字")
```

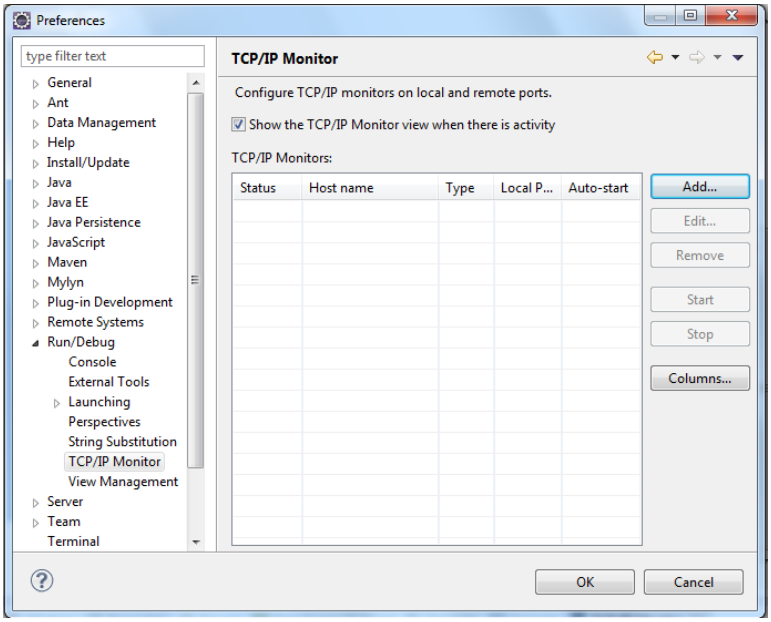
Troubleshooting

Traces enable the recovery of the XML input in order to process the case in SoapUI again, for example, and the XML output to get the entire error message. This paragraph describes how to activate the traces.

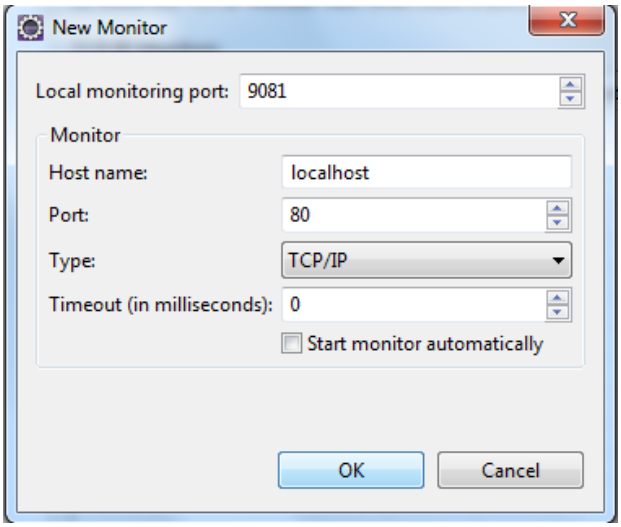
To enable the display of the traces, a server must be created from Eclipse that will act as a TCP/IP monitor.

In the **Window/Preferences**, select **Run/Debug** and **TCP/IP Monitor**.

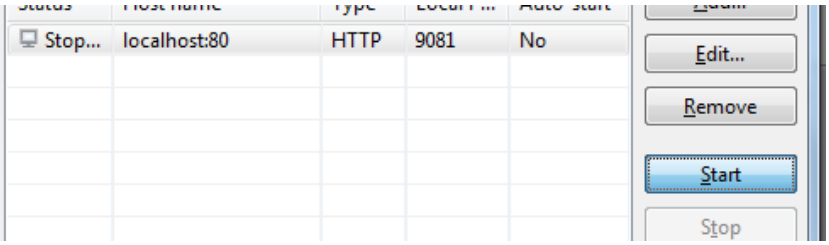
Then click on **Add**.



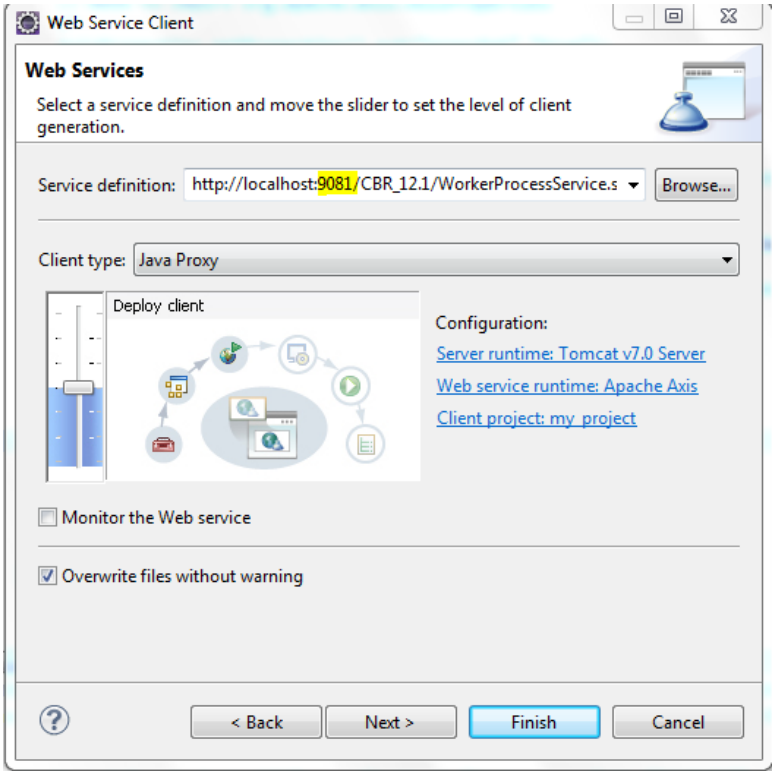
Specify the same parameter as on the illustration below, and replace "localhost" by the name of the Web Service domain; then click on **OK**.



Click on the monitor just created, click on **Start** to launch, and then on **OK**.

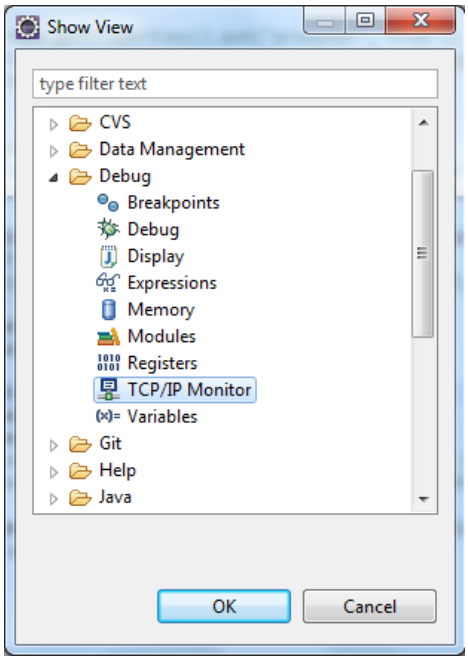


You must now create a Web Servicve client and specify the port of the monitor (9081) in the URL

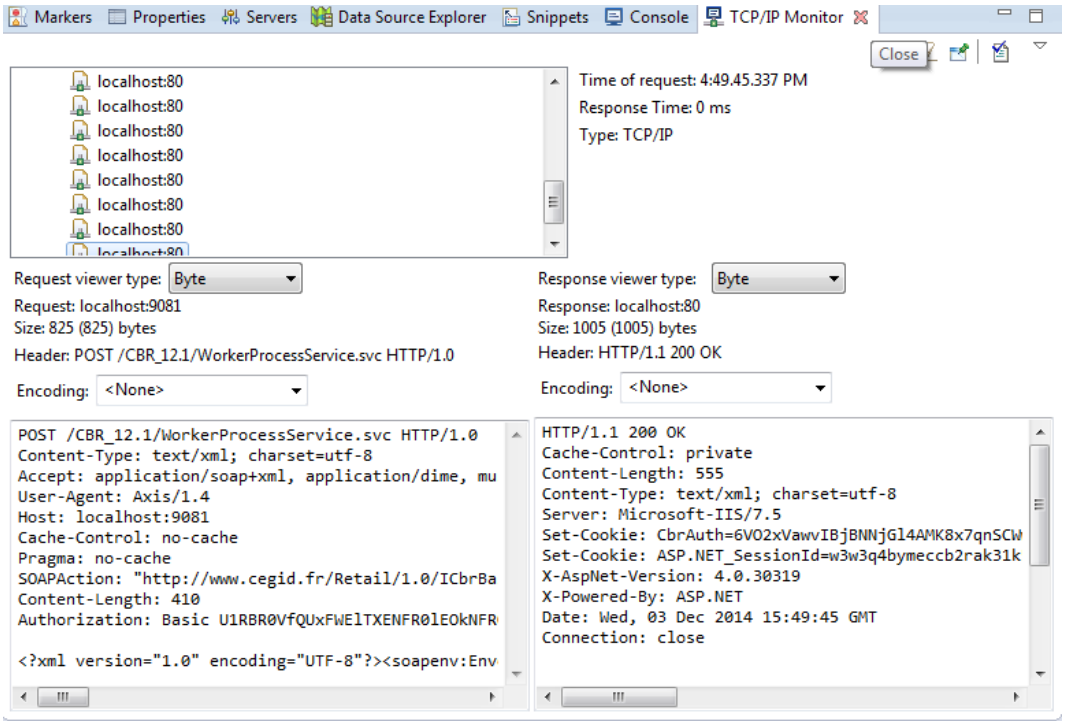


Display the **TCP/IP Monitor** view via **Window**/**Show view**/**Other...**

And under **Debug**, select **TCP/IP Monitor**



Then you just have to consume the Web Service from Eclipse to get the traces in the **TCP/IP Monitor** window.



Example of traces:

request.txt

response.txt

8. JavaScript

Installing the environment

- Use your current browser or install (Google Chrome)
- Original sources to use soap access service from JavaScript are available here: JavaScriptSoapClient
- We use ObjTree to build the xml request, original sources are available here: ObjTree

Disclaimer

Do not use original sources of soapclient and objtree because they doesn't support our api configuration. (Forced domain namespace, json tree with pluralization)

We have edited soapclient.js and objtree.js to make it work with our api

- We added the ability to force namespace and hidden action in wsdl
- We added a new parser to generate XML data from Json
- We commented all unused sources from original file

You can use our scripts instead:

soapclient.js

objtree.js

Note: These scripts are examples. In production, the code may be different. Cegid may not be liable for them use.

Basic authentication

Code

Script used to consume the Web Service with a Basic authentication.

basic_auth.html

Sample of code and description:

Code	Info

`<script type="text/javascript" src="./soapclient.js"></script>`
`<script type="text/javascript" src="./objtree.js" ></script>`
`<script type="text/javascript">`
`// Request, it's only a json tree`
`var paramCustomer = {`
`text: "ttt",`
`clientContext: {`
`DatabaseId: "CBR_AUTO" // Database ID`
`}`
`};`
	Include soapClient script
	Include objtree script
	
	Prepare the request

`function SoapService() {`
`setCredentials(`
`"USERNAME", // Username`
`"PASSWORD", // Password`
`"http://URL_SERVER/Retail_14.0/", // Url of the server,`
`"Basic" // Authentication method Basic or NTLM`
`);`
	Database name, identifier and password
	URL of the Web Service, authentication method

`callService(`
`"WorkerProcessService.svc", // Name of the webservice`
`"HelloWorld", // Method`
`resultHello, // Callback`
`paramCustomer, // Parameters`
`"http://www.cegid.fr/Retail/1.0", // Domain`
 	
`"http://www.cegid.fr/Retail/1.0/ICbrBasicWebServiceInterface/HelloWorld" // Action`
`);`
`}`
	Call the HelloWorld method

`// Callback result`
`function resultHello(o, req) {`
`try {`
`console.log(req); // Display result in console`
`document.getElementById("result").innerText = new XMLSerializer().serializeToString(req);`
 	
`} catch (e) {`
`document.getElementById("result").innerText = "Error, open your console"`
`}`
`}`
`window.onload = SoapService;`
`</script>`
	Display results in console and text

The created request corresponds to the following XML:

```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <HelloWorld xmlns="http://www.cegid.fr/Retail/1.0">
      <text>ttt</text>
      <clientContext>
        <DatabaseId>CBR_AUTO</DatabaseId>
      </clientContext>
    </HelloWorld>
  </soap:Body>
</soap:Envelope>
```

Execution of the script

Open index.hml, the result of the request is displayed in text, you can also show console to display result

Response

The program displays the response from the Web Service, specifying the type of authentication used: Basic

```
Now: (2017-04-10T16:48:07.8361055)
ApplicationBase: (C:\inetpub\wwwroot\Cegid Retail\Y2 14.0\)
InputText: (ttt)
DataBaseId: (DATABASE)
ErpIdentity: (USR) (USER) (DOMAIN)
Current Identity: (DOMAIN\USER) (Basic)
CurrentCulture: (fr-FR)
CurrentUICulture: (fr-FR)
CBR Version: (14.0.0.800)
```

Complex case (AddNewCustomer)

How can you create more complex types? See example with AddNewCustomer:

AddNewCustomer.
html

Parameters can be passed by a json tree

Arrays

Create an array with a json array;

JSON parameter	XML result

`UserDefinedBooleans: {`
`  UserDefinedBoolean: [`
`  { Id: 1, Value: true },`
`  { Id: 2, Value: false`
`  },`
`  { Id: 3, Value: true }`
`  ]`
`}`
	`<UserDefinedBooleans>`
	`  <UserDefinedBoolean>`
	`  <Id>1</Id>`
	`  <Value>true</Value>`
	`  </UserDefinedBoolean>`
	`  <UserDefinedBoolean>`
	`  <Id>2</Id>`
	`  <Value>false</Value>`
	`  </UserDefinedBoolean>`
	`  <UserDefinedBoolean>`
	`  <Id>3</Id>`
	`  <Value>true</Value>`
	`  </UserDefinedBoolean>`
	`</UserDefinedBooleans>`

Dates

To create a DateTime, specify the date and time, then convert it to ISO String to format to DateTime

```
var date = new Date().toISOString();
```

JSON parameter	XML result

`UserDefinedDates: {`
`  UserDefinedDate: [`
`  { Id: 1, Value: date },`
`  { Id: 2, Value: date },`
`  { Id: 3, Value: date }`
`  ]`
`}`
	`<UserDefinedDates>`
	`  <UserDefinedDate>`
	`  <Id>1</Id>`
	`  <Value>2017-04-12T12:41:29.127Z</Value>`
	`  </UserDefinedDate>`
	`  <UserDefinedDate>`
	`  <Id>2</Id>`
	`  <Value>2017-04-12T12:41:29.127Z</Value>`
	`  </UserDefinedDate>`
	`  <UserDefinedDate>`
	`  <Id>3</Id>`
	`  <Value>2017-04-12T12:41:29.127Z</Value>`
	`  </UserDefinedDate>`
	`</UserDefinedDates>`

Enum

There is no need to enum in JSON, so go directly to the constant value

```
{ customerData: { AddressData: { CountryIdType: "Internal" } } };
```

Non ASCII characters

Do not forget to save the script with UTF-8 encoding.

```
{ Id: 2, Value: "漢字" }
```

Troubleshooting

Traces are available in the console of your browser, you can also use Fiddler to debug.

9. Authentication cookie

Introduction

To avoid authenticating each time you call the Web Service you can implement a mechanism based on cookies used for the Basic HTTP. This system optimizes the WS/HTTP flow.

Therefore, you will prompt the resource without authentication, but with the cookie. If the server responds 401 (Unauthorized), you will prompt the resource again but WITH authentication and the new authentication cookie will be stored for future calls.

Header of the request with **authentication**:

```
POST /CBR_12.1/WorkerProcessService.svc HTTP/1.1
Host: localhost
Connection: Keep-Alive
User-Agent: PHP-SOAP/5.6.3
Content-Type: text/xml; charset=utf-8
SOAPAction: "http://www.cegid.fr/Retail/1.0/ICbrBasicWebServiceInterface/HelloWorld"
Content-Length: 354
Authorization: Basic U1RBR0VfQUxFWElTXENFR0lEOkNFR0lE
```

Header of the request with the **cookie**:

```
POST /CBR_12.1/WorkerProcessService.svc HTTP/1.1
Host: localhost
Connection: Keep-Alive
User-Agent: PHP-SOAP/5.6.3
Content-Type: text/xml; charset=utf-8
SOAPAction: "http://www.cegid.fr/Retail/1.0/ICbrBasicWebServiceInterface/HelloWorld"
Content-Length: 354
Cookie:
CbrAuth=6I2djBiRTyfhRrW/4+yUvwwF/64p/zHCiJ3ZeEWYA3YW0WqJElsHgs/KORav60mFznmPXk0JUjSQeIWEcqCVFTbyu
fwnVp6WhaCih71ZILw=;
```

Sample of a PHP script

CookieSoap

You will use the CookieSoap.php file that contains the functions used to read and write in the file that stores the cookie.

CookieSoap.php

The file contains two functions:

- WriteCookie() that stores the value passed as parameter in a "cookie" file.
- ReadCookie() that returns the value contained in a "cookie" file.

The file containing the authentication cookie is named "cookie" and is created in the same directory as the script. To change the name and the path of the file, you just have to change the value of the `$filename` variable in the CookieSoap.php file.

Code

Test of the HelloWorld method of the WorkerProcessService Web Service with the use of the authentication cookie.

HelloWorld_cookie.php

Sample of code and description:

Code	Info

```php
<?php
  include('CookieSoap.php');

  $url = "http://localhost/CBR_12.1/WorkerProcessService.svc?singleWsdl";

  $client = new SoapClient($url);

  $request = new StdClass();
  $request->text = "test";
  $request->clientContext = new StdClass();
  $request->clientContext->DatabaseId = "DATABASE";

  $client->__setCookie("CbrAuth", ReadCookie());

  try {
    $resu = $client->HelloWorld($request);
  } catch (Exception $e) {
    if ($e->getMessage() == "Unauthorized")
    {
      $client = new SoapClient($url,
        array(
          "login" => "DOMAIN\\USERNAME",
          "password" => "PASSWORD"
        )
      );
    }

    $resu = $client->HelloWorld($request);

    WriteCookie($client->_cookies["CbrAuth"][0]);
  }

  print_r($resu);
?>
```

Include CookieSoap

Link to the WSDL file

Create the client

Prepare the request

Recover the cookie

Call the HelloWorld method

If the server responds 401, you will create a client with authentication

Database name, identifier and password

Call the HelloWorld method
Store the authentication cookie

Display the result

10. Appendix

Content of each file mentioned in this document.

basic_auth.php

```
<?php
  /*
  ** Download PHP from http://windows.php.net/download/
  ** Copy in C:\php
  **
  ** Copy c:\php\php.ini-production into c:\php\php.ini and uncomment these lines (remove the
semicolon) :
  ** ;extension_dir = "ext"
  ** ;extension=php_openssl.dll
  ** ;extension=php_soap.dll
  */

  // Web Service URL
  $url = "http://localhost/CBR_12.1/WorkerProcessService.svc";

  // client proxy
  $client = new SoapClient( $url . "?singleWsdl",
    array(
      "location" => $url,
      "login" => "DOMAIN\\USERNAME",
      "password" => "PASSWORD"
      )
  );

  // request complex type
  $request = new StdClass();
  $request->text = "ttt";
  $request->clientContext = new StdClass();
  $request->clientContext->DatabaseId = "DATABASE";

  // call
  $resu = $client->HelloWorld($request);
  print_r( $resu );
?>
```

AddNewCustomer.php

```
<?php
  /*
  ** Download PHP from http://windows.php.net/download/
  ** Copy in C:\php
  **
  ** Copy c:\php\php.ini-production into c:\php\php.ini and uncomment these lines (remove the
semicolon) :
  ** ;extension_dir = "ext"
  ** ;extension=php_openssl.dll
  ** ;extension=php_soap.dll
  */

  // client proxy
  $client = new SoapClient("http://localhost/CBR_12.1/CustomerWcfService.svc?singleWsdl",
    array(
      "location" => "http://localhost/CBR_12.1/CustomerWcfService.svc",
      "login" => "DOMAIN\\USERNAME",
      "password" => "PASSWORD"
      )
  );

  // request complex type
  $request = new StdClass();

  $request->customerData = new StdClass();
```

```php
$request->customerData->AddressData = new StdClass();
$request->customerData->AddressData->AddressLine1 = "23 rue foobar";
$request->customerData->AddressData->City = "Paris";
$request->customerData->AddressData->CountryId = "FRA";
$request->customerData->AddressData->CountryIdType = "Internal";
$request->customerData->AddressData->Nata = false;
$request->customerData->AddressData->ZipCode = "42000";

$request->customerData->EmailData = new StdClass();
$request->customerData->EmailData->EmailingAccepted = false;

$request->customerData->FirstName = "John";
$request->customerData->IsCompany = false;
$request->customerData->LastName = "Doe";
$request->customerData->TitleId = "MR";

// Create some UserDefinedBoolean...
$UserDefinedBoolean1 = new StdClass();
$UserDefinedBoolean1->Id = 1;
$UserDefinedBoolean1->Value = true;
$UserDefinedBoolean2 = new StdClass();
$UserDefinedBoolean2->Id = 2;
$UserDefinedBoolean2->Value = false;
$UserDefinedBoolean3 = new StdClass();
$UserDefinedBoolean3->Id = 3;
$UserDefinedBoolean3->Value = true;

// ...And create the UserDefinedBooleans array with UserDefinedBoolean objects
$request->customerData->UserDefinedBooleans = array($UserDefinedBoolean1, $UserDefinedBoolean2,
$UserDefinedBoolean3);

$UserDefinedDate1 = new StdClass();
$UserDefinedDate1->Id = 1;

// Create the Datetime object
$date = new Datetime(null, new DateTimeZone("UTC"));
$date->SetDate(1970, 01, 01); // SetDate(Year, Month, Day)
$date->SetTime(18, 00, 00); // SetTime(Hour, Minute, Second)
// Convert the object to a string in order to assigned it to Value
$UserDefinedDate1->Value = $date->format("Y-m-d\TH:i:s");

$request->customerData->UserDefinedDates = array($UserDefinedDate1);

$UserDefinedText1 = new StdClass();
$UserDefinedText1->Id = 1;
$UserDefinedText1->Value = "some text...";
$UserDefinedText2 = new StdClass();
$UserDefinedText2->Id = 2;
$UserDefinedText2->Value = "漢字"; // non ascii characters works perfectly
$request->customerData->UserDefinedTexts = array($UserDefinedText1, $UserDefinedText2);

$UserDefinedValue1 = new StdClass();
$UserDefinedValue1->Id = 1;
$UserDefinedValue1->Value = 0;
$request->customerData->UserDefinedValues = array($UserDefinedValue1);

$request->customerData->UsualStoreId = "302";

$request->customerData->BirthDateData = new StdClass();
$request->customerData->BirthDateData->BirthDateDay = 1;
$request->customerData->BirthDateData->BirthDateMonth = 1;
$request->customerData->BirthDateData->BirthDateYear = 1970;

$request->customerData->CurrencyId = "EUR";
$request->customerData->LanguageId = "FRA";
$request->customerData->NationalityId = "FRA";

$request->customerData->OptinAlternativeEmail = "AskCustomer";
$request->customerData->OptinEmail = "AskCustomer";
$request->customerData->OptinMobile = "AskCustomer";
$request->customerData->OptinOfficePhone = "AskCustomer";
$request->customerData->OptinPostal = "AskCustomer";
```

```
  $request->customerData->Sex = "M";
  $request->customerData->ShortName = "DOE";
  $request->customerData->VATSystem = "FRA";
  $request->customerData->ValidAlternativeEmail = true;
  $request->customerData->ValidEmail = true;
  $request->customerData->ValidMobile = true;
  $request->customerData->ValidOfficePhone = true;

  $request->clientContext = new StdClass();
  $request->clientContext->DatabaseId = "DATABASE";

  // call
  $resu = $client->AddNewCustomer($request);
  print_r( $resu );
?>
```

soapDebug.php

```
<?php
function soapDebug($client){
   $requestHeaders = str_ireplace('><', ">\r\n<", $client->__getLastRequestHeaders()) . "\r\n";
   $request = str_ireplace('><', ">\r\n<", $client->__getLastRequest()) . "\r\n";
   $responseHeaders = str_ireplace('><', ">\r\n<", $client->__getLastResponseHeaders()) . "\r\n";
   $response = str_ireplace('><', ">\r\n<", $client->__getLastResponse()) . "\r\n";

   echo "REQUEST HEADERS:\r\n" . $requestHeaders;
   echo "REQUEST:\r\n" . $request;
   echo "RESPONSE HEADERS:\r\n" . $responseHeaders;
   echo "RESPONSE:\r\n" . $response;
}
?>
```

test_logs.php

```
<?php
  include('soapDebug.php');
  // client proxy
  $client = new SoapClient("http://localhost/CBR_12.1/WorkerProcessService.svc?singleWsdl",
    array(
      "login" => "STAGE_ALEXIS\\CEGID",
      "password" => "CEGID",
     "trace" => true
      )
  );

  // request complex type
  $request = new StdClass();
  $request->text = "test";
  $request->clientContext = new StdClass();
  $request->clientContext->DatabaseId = "STAGE_ALEXIS";

  // call
  try {
   $resu = $client->HelloWorld($request);
  } catch (Exception $e) {}
  soapDebug($client);
?>
```

basic_auth.cs

```
/*
** Create the client proxy by adding the Service Reference from the project
*/

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.ServiceModel;
using basic_auth.ws; // using project_name.namespace (where namespace is the name given to the
Service Reference)
```

```
namespace basic_auth // Rename it by the name of the project
{
    class Program
    {
        static void Main(string[] args)
        {
                    // Use BasicHttpSecurityMode.Transport instead of
BasicHttpSecurityMode.TransportCredentialOnly for HTTPS
            var bhb = new BasicHttpBinding(BasicHttpSecurityMode.TransportCredentialOnly);
                    var auth = HttpClientCredentialType.Basic;
            bhb.Security.Transport.ClientCredentialType = auth;

            // Replace WorkerProcessServiceClient with the name of your web service
            var service = new WorkerProcessServiceClient(
                bhb,
                new EndpointAddress("http://localhost/CBR_12.1/WorkerProcessService.svc")
            );

            service.ClientCredentials.UserName.UserName = "DOMAIN\\USERNAME";
            service.ClientCredentials.UserName.Password = "PASSWORD";

            // Initiliaze the HelloWorld arguments
            var text = "ttt";
            var clientContext = new RetailContext();
            clientContext.DatabaseId = "DATABASE";

            // Make the web service call, replace HelloWorld with the name of your method
            var result = service.HelloWorld(text, clientContext);

            // Display the result
            Console.WriteLine(result);
            Console.ReadLine();
        }
    }
}
```

AddNewCustomer.cs

```
/*
** Create the client proxy by adding the Service Reference from the project
*/

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.ServiceModel;
using AddNewCustomer.ws; // using project_name.namespace (where namespace is the name given to
the Service Reference)

namespace AddNewCustomer // Rename it by the name of the project
{
    class Program
    {
        static void Main(string[] args)
        {
                    // Use BasicHttpSecurityMode.Transport instead of
BasicHttpSecurityMode.TransportCredentialOnly for HTTPS
            var bhb = new BasicHttpBinding(BasicHttpSecurityMode.TransportCredentialOnly );
            bhb.Security.Transport.ClientCredentialType = HttpClientCredentialType.Basic;
            bhb.Security.Message.ClientCredentialType = BasicHttpMessageCredentialType.UserName;

            // Replace CustomerWcfService with the name of your web service
            var service = new CustomerWcfServiceClient(
                bhb,
                new EndpointAddress("http://localhost/CBR_12.1/CustomerWcfService.svc")
            );

            service.ClientCredentials.UserName.UserName = "DOMAIN\\USERNAME";
            service.ClientCredentials.UserName.Password = "PASSWORD";

            // Initiliaze the AddNewCustomer arguments
```

```
            var customerData = new CustomerInsertData();

            customerData.AddressData = new AddressDataType();
            customerData.AddressData.AddressLine1 = "23 rue foobar";
            customerData.AddressData.City = "Paris";
            customerData.AddressData.CountryId = "FRA";
            customerData.AddressData.CountryIdType = CountryIdType.Internal;
            customerData.AddressData.Nata = false;
            customerData.AddressData.ZipCode = "42000";

            customerData.EmailData = new EmailDataType();
            customerData.EmailData.EmailingAccepted = false;

            customerData.FirstName = "John";
            customerData.IsCompany = false;
            customerData.LastName = "Doe";
            customerData.TitleId = "MR";

            customerData.UsualStoreId = "302";

            customerData.BirthDateData = new BirthDateDataType();
            customerData.BirthDateData.BirthDateDay = 1;
            customerData.BirthDateData.BirthDateMonth = 1;
            customerData.BirthDateData.BirthDateYear = 1970;

            customerData.CurrencyId = "EUR";
            customerData.LanguageId = "FRA";
            customerData.NationalityId = "FRA";

            customerData.OptinAlternativeEmail = CustomerInformationType.AskCustomer;
            customerData.OptinEmail = CustomerInformationType.AskCustomer;
            customerData.OptinMobile = CustomerInformationType.AskCustomer;
            customerData.OptinOfficePhone = CustomerInformationType.AskCustomer;
            customerData.OptinPostal = CustomerInformationType.AskCustomer;

                   // Create a list of UserDefinedBoolean...
            List<UserDefinedBoolean> userDefinedBooleanList = new List<UserDefinedBoolean>()
            {
                new UserDefinedBoolean(){ Id = UserDefinedId._1, Value = true },
                new UserDefinedBoolean(){ Id = UserDefinedId._2, Value = false },
                new UserDefinedBoolean(){ Id = UserDefinedId._3, Value = true }
            };
                   // ... And convert it to an array (UserDefinedBooleans is an array, not a
list)
            customerData.UserDefinedBooleans = userDefinedBooleanList.ToArray();

            List<UserDefinedDate> userDefinedDateList = new List<UserDefinedDate>()
            {
                            // Dates are created as follow : var date = new DateTime(year, month,
day, hour, minute, second)
                new UserDefinedDate(){ Id = UserDefinedId._1, Value = new DateTime(1970, 01, 01,
18, 00, 00) }
            };
            customerData.UserDefinedDates = userDefinedDateList.ToArray();

            List<UserDefinedText> userDefinedTextList = new List<UserDefinedText>()
            {
                new UserDefinedText(){ Id = UserDefinedId._1, Value = "some text..." },
                new UserDefinedText(){ Id = UserDefinedId._2, Value = "漢字" } // non ascii
characters works perfectly
            };
            customerData.UserDefinedTexts = userDefinedTextList.ToArray();

            customerData.Sex = "M";
            customerData.ShortName = "DOE";
            customerData.VATSystem = "FRA";
            customerData.ValidAlternativeEmail = true;
            customerData.ValidEmail = true;
            customerData.ValidMobile = true;
            customerData.ValidOfficePhone = true;

            var clientContext = new RetailContext();
            clientContext.DatabaseId = "DATABASE";
```

```
            // Make the web service call, replace AddNewCustomer with the name of your method
            var result = service.AddNewCustomer(customerData, clientContext);

            // Display the result
            Console.WriteLine(result);
            Console.ReadLine();
        }
    }
}
```

basic_auth.py

```
# Python 2.7 is used, it can be download here : https://www.python.org/downloads/

# 2 packages are required :
# https://pypi.python.org/packages/source/s/setuptools/setuptools-1.4.2.tar.gz
# https://pypi.python.org/packages/source/s/suds/suds-0.4.tar.gz

import suds

# Replace with the Web Service URL
url = 'http://localhost/CBR_12.1/WorkerProcessService.svc?wsdl'

username = "DOMAIN\\USER"
password = "PASSWORD"

# Create the client of the Web Service
client = suds.client.Client(url, username=username, password=password)

# Create the RetailContext object needed by the method 'HelloWorld'
clientContext = client.factory.create('ns2:RetailContext')

text = "test"
clientContext.DatabaseId = "DATABASE"

# Retrieve the HelloWorld's response
response = client.service.HelloWorld(text, clientContext)

# Display the result
print response
```

AddNewCustomer.py

```
# Python 2.7 is used, it can be download here : https://www.python.org/downloads/

# 2 packages are required :
# https://pypi.python.org/packages/source/s/setuptools/setuptools-1.4.2.tar.gz
# https://pypi.python.org/packages/source/s/suds/suds-0.4.tar.gz

# And this one if NTLM authentication is used :
# https://pypi.python.org/packages/source/p/python-ntlm/python-ntlm-1.1.0.tar.gz

import suds
import datetime

# Replace with the Web Service URL
url = 'http://localhost/CBR_12.1/CustomerWcfService.svc?wsdl'

username = "DOMAIN\\USERNAME"
password = "PASSWORD"

# Create the client of the Web Service
client = suds.client.Client(url, username=username, password=password)

customerData = client.factory.create('ns2:CustomerInsertData')

customerData.AddressData = client.factory.create('ns2:AddressDataType')
customerData.AddressData.AddressLine1 = "23 rue foobar"
customerData.AddressData.City = "Paris"
customerData.AddressData.CountryId = "FRA"
customerData.AddressData.CountryIdType = "Internal"
customerData.AddressData.Nata = False
customerData.AddressData.ZipCode = "42000"
```

```
customerData.EmailData = client.factory.create('ns2:EmailDataType')
customerData.EmailData.EmailingAccepted = False

customerData.FirstName = "John"
customerData.IsCompany = False
customerData.LastName = "Doe"
customerData.TitleId = "MR"

customerData.UserDefinedBooleans = client.factory.create('ns2:ArrayOfUserDefinedBoolean')

# Create the UserDefinedBooleans "cells"...
UserDefinedBoolean1 = client.factory.create('ns2:UserDefinedBoolean')
UserDefinedBoolean1.Id = 1
UserDefinedBoolean1.Value = True

UserDefinedBoolean2 = client.factory.create('ns2:UserDefinedBoolean')
UserDefinedBoolean2.Id = 2
UserDefinedBoolean2.Value = False

UserDefinedBoolean3 = client.factory.create('ns2:UserDefinedBoolean')
UserDefinedBoolean3.Id = 3
UserDefinedBoolean3.Value = True

# ... And append them to the UserDefinedBooleans (this will create an array of
UserDefinedBoolean)
customerData.UserDefinedBooleans.UserDefinedBoolean.append(UserDefinedBoolean1)
customerData.UserDefinedBooleans.UserDefinedBoolean.append(UserDefinedBoolean2)
customerData.UserDefinedBooleans.UserDefinedBoolean.append(UserDefinedBoolean3)

customerData.UserDefinedDates = client.factory.create('ns2:ArrayOfUserDefinedDate')

UserDefinedDate1 = client.factory.create('ns2:UserDefinedDate')
UserDefinedDate1.Id = 1
# datetime.datetime(Year, Month, Day, Hour, Minute, Second)
UserDefinedDate1.Value = datetime.datetime(1970, 01, 01, 18, 00, 00)

customerData.UserDefinedDates.UserDefinedDate.append(UserDefinedDate1)

customerData.UserDefinedTexts = client.factory.create('ns2:ArrayOfUserDefinedText')
UserDefinedText1 = client.factory.create('ns2:UserDefinedText')
UserDefinedText1.Id = 1
UserDefinedText1.Value = "some text..."
UserDefinedText2 = client.factory.create('ns2:UserDefinedText')
UserDefinedText2.Id = 2
UserDefinedText2.Value = unicode("漢字", "utf-8") # non ascii characters must be encoded as UTF-8
customerData.UserDefinedTexts.UserDefinedText.append(UserDefinedText1)
customerData.UserDefinedTexts.UserDefinedText.append(UserDefinedText2)

customerData.UserDefinedValues = client.factory.create('ns2:ArrayOfUserDefinedValue')
UserDefinedValue1 = client.factory.create('ns2:UserDefinedValue')
UserDefinedValue1.Id = 1
UserDefinedValue1.Value = 0
customerData.UserDefinedValues.UserDefinedValue.append(UserDefinedValue1)

customerData.UsualStoreId = "302"

customerData.BirthDateData = client.factory.create('ns2:BirthDateDataType')
customerData.BirthDateData.BirthDateDay = 1
customerData.BirthDateData.BirthDateMonth = 1
customerData.BirthDateData.BirthDateYear = 1970

customerData.CurrencyId = "EUR"
customerData.LanguageId = "FRA"
customerData.NationalityId = "FRA"

customerData.OptinAlternativeEmail = "AskCustomer"
customerData.OptinEmail = "AskCustomer"
customerData.OptinMobile = "AskCustomer"
customerData.OptinOfficePhone = "AskCustomer"
customerData.OptinPostal = "AskCustomer"

customerData.Sex = "M"
customerData.ShortName = "DOE"
```

```
customerData.VATSystem = "FRA"
customerData.ValidAlternativeEmail = True
customerData.ValidEmail = True
customerData.ValidMobile = True
customerData.ValidOfficePhone = True

clientContext = client.factory.create('ns2:RetailContext')
clientContext.DatabaseId = "DATABASE"

# Retrieve the AddNewCustomer's response
response = client.service.AddNewCustomer(customerData, clientContext)

# Display the result
print response.encode("utf-8") # if there is non ascii characters in the SOAP request, the
response must be encode as UTF-8
```

test_logs.py

```
# Python 2.7 is used, it can be download here : https://www.python.org/downloads/

# 2 packages are required :
# https://pypi.python.org/packages/source/s/setuptools/setuptools-1.4.2.tar.gz
# https://pypi.python.org/packages/source/s/suds/suds-0.4.tar.gz

import suds
import logging

# Replace with the Web Service URL
url = 'http://localhost/CBR_12.1/WorkerProcessService.svc?wsdl'

username = "STAGE_ALEXIS\\CEGID"
password = "CEGID"

# Create the client of the Web Service
client = suds.client.Client(url, username=username, password=password)

logging.basicConfig(level=logging.INFO)
logging.getLogger('suds.client').setLevel(logging.DEBUG)

# Create the RetailContext object needed by the method 'HelloWorld'
clientContext = client.factory.create('ns2:RetailContext')

text = "test"
clientContext.DatabaseId = "STAGE_ALEXIS"

# Retrieve the HelloWorld's response
response = client.service.HelloWorld(text, clientContext)
```

basic_auth.rb

```
# Ruby can be downloaded here : http://rubyinstaller.org/downloads/

# The script is using savon, it can be installed by running this command line :
# gem install savon

require 'savon'

# Create client proxy
client = Savon.client do
   wsdl "http://localhost/CBR_12.1/WorkerProcessService.svc?wsdl"
   basic_auth("DOMAIN\\USERNAME", "PASSWORD")
   namespace "http://www.cegid.fr/Retail/1.0"
end

# Create message
message ={
   "wsdl:text"=>"test",
   "wsdl:clientContext"=>{
           "wsdl:DatabaseId"=>"DATABASE"
   }
}

# Call the web service
```

```
response = client.call(:hello_world, message: message)

# Display the response
puts response.to_xml.gsub("&#xD;", "")
```

AddNewCustomer.rb

```
# Ruby can be downloaded here : http://rubyinstaller.org/downloads/

# The script is using savon, it can be installed by running this command line :
# gem install savon

require 'savon'

# Create client proxy
client = Savon.client do
   wsdl "http://localhost/CBR_12.1/CustomerWcfService.svc?wsdl"
   basic_auth("DOMAIN\\USERNAME", "PASSWORD")
   namespace "http://www.cegid.fr/Retail/1.0"
end

# Create message
message ={
   "wsdl:customerData"=>{
           "wsdl:AddressData"=>{
                   "wsdl:AddressLine1"=>"23 rue foobar",
                   "wsdl:City"=>"Paris",
                   "wsdl:CountryId"=>"FRA",
                   "wsdl:CountryIdType"=>"Internal",
                   "wsdl:Nata"=>false,
                   "wsdl:ZipCode"=>"42000"
                   },
           "wsdl:EmailData"=>{
                   "wsdl:EmailingAccepted"=>false
                   },
           "wsdl:FirstName"=>"John",
           "wsdl:IsCompany"=>false,
           "wsdl:LastName"=>"Doe",
           "wsdl:TitleId"=>"MR",
           "wsdl:UserDefinedBooleans"=>{
                   # Create an array of UserDefinedBoolean (see the '[]' insted of '{}')
                   "wsdl:UserDefinedBoolean"=>[
                           {
                                   "wsdl:Id"=>1,
                                   "wsdl:Value"=>true
                           },
                           {
                                   "wsdl:Id"=>2,
                                   "wsdl:Value"=>false
                           },
                           {
                                   "wsdl:Id"=>3,
                                   "wsdl:Value"=>true
                           }
                           ]
                   },
           "wsdl:UserDefinedDates"=>{
                   "wsdl:UserDefinedDate"=>[
                           {
                                   "wsdl:Id"=>1,
                                   # DateTime.new(Year, Month, Day, Hour, Minute, Second)
                                   "wsdl:Value"=>DateTime.new(1970, 01, 01, 18, 00,
00).strftime("%Y-%m-%dT%H:%M:%S") # strftime convert the DateTime to a String
                           }
                           ]
                   },
           "wsdl:UserDefinedTexts"=>{
                   "wsdl:UserDefinedText"=>[
                           {
                                   "wsdl:Id"=>1,
                                   "wsdl:Value"=>"some text..."
                           },
                           {
```

```
                                        "wsdl:Id"=>2,
                                        "wsdl:Value"=>"漢字" # non ascii characters works perfectly
                                }
                                ]
                        },
                "wsdl:UserDefinedValues"=>{
                        "wsdl:UserDefinedValue"=>{
                                "wsdl:Id"=>1,
                                "wsdl:Value"=>0
                                }
                        },
                "wsdl:UsualStoreId"=>"302",
                "wsdl:BirthDateData"=>{
                        "wsdl:BirthDateDay"=>1,
                        "wsdl:BirthDateMonth"=>1,
                        "wsdl:BirthDateYear"=>1970
                        },
                "wsdl:CurrencyId"=>"EUR",
                "wsdl:LanguageId"=>"FRA",
                "wsdl:NationalityId"=>"FRA",
                "wsdl:OptinAlternativeEmail"=>"AskCustomer",
                "wsdl:OptinEmail"=>"AskCustomer",
                "wsdl:OptinMobile"=>"AskCustomer",
                "wsdl:OptinOfficePhone"=>"AskCustomer",
                "wsdl:OptinPostal"=>"AskCustomer",
                "wsdl:Sex"=>"M",
                "wsdl:ShortName"=>"DOE",
                "wsdl:VATSystem"=>"FRA",
                "wsdl:ValidAlternativeEmail"=>true,
                "wsdl:ValidEmail"=>true,
                "wsdl:ValidMobile"=>true,
                "wsdl:ValidOfficePhone"=>true
        },
        "wsdl:clientContext"=>{
                "wsdl:DatabaseId"=>"DATABASE"
        }
}

# Call the web service
response = client.call(:add_new_customer, message: message)

# Display the response
puts response.to_xml.gsub("&#xD;", "")
#print response.body[:add_new_customer_response][:add_new_customer_result]
```

test_logs.rb

```
require 'savon'

# Create client proxy
client = Savon.client do
   wsdl "http://localhost:81/CBR_12.1_Basic/WorkerProcessService.svc?wsdl"
   basic_auth("STAGE_ALEXIS\\CEGID", "CEGID")
   namespace "http://www.cegid.fr/Retail/1.0"
   log true
   pretty_print_xml true
end

# Create message
message ={
   "wsdl:text"=>"test",
   "wsdl:clientContext"=>{
           "wsdl:DatabaseId"=>"STAGE_ALEXIS"
   }
}

# Call the web service
response = client.call(:hello_world, message: message)
```

basic_auth.java

```
/*
** Apache Tomcat 7.0 and Axis2 are required to run this code
```

```java
**
** Create the client proxy by creating a new Web Service Client in the project
*/

import org.tempuri.BasicHttpBinding_IWorkerProcessServiceStub;
import org.tempuri.WorkerProcessServiceLocator;
import fr.cegid.www.Retail._1_0.*;

public class basic_auth {

   public static void main(String[] args) throws Exception {
           WorkerProcessServiceLocator ws = new WorkerProcessServiceLocator();
           BasicHttpBinding_IWorkerProcessServiceStub stub =
(BasicHttpBinding_IWorkerProcessServiceStub)ws.getBasicHttpBinding_IWorkerProcessService();

           stub.setUsername("DOMAIN\\USERNAME");
           stub.setPassword("PASSWORD");

           String text = "test";
           RetailContext clientContext = new RetailContext();
           clientContext.setDatabaseId("DATABASE");
           // We can also give the DatabaseId to directly to the constructor :
           // RetailContext clientContext = new RetailContext("DATABASE");

           // Call
           String response = stub.helloWorld(text, clientContext);

           // Print response
           System.out.println(response);
   }

}
```

AddNewCustomer.java

```java
/*
** Apache Tomcat 7.0 and Axis2 are required to run this code
**
** Create the client proxy by creating a new Web Service Client in the project
*/

import java.util.Calendar;
import org.tempuri.BasicHttpBinding_ICustomerWcfServiceStub;
import org.tempuri.CustomerWcfServiceLocator;
import fr.cegid.www.Retail._1_0.*;

public class AddNewCustomer {

   public static void main(String[] args) throws Exception {
           CustomerWcfServiceLocator ws = new CustomerWcfServiceLocator();
           BasicHttpBinding_ICustomerWcfServiceStub stub =
(BasicHttpBinding_ICustomerWcfServiceStub)ws.getBasicHttpBinding_ICustomerWcfService();

           stub.setUsername("DOMAIN\\USERNAME");
           stub.setPassword("PASSWORD");

           CustomerInsertData customerData = new CustomerInsertData();

           AddressDataType addressData = new AddressDataType();
           addressData.setAddressLine1("23 rue foobar");
           addressData.setCity("Paris");
           addressData.setCountryId("FRA");
           addressData.setCountryIdType(CountryIdType.Internal);
           addressData.setNata(false);
           addressData.setZipCode("42000");
           customerData.setAddressData(addressData);

           EmailDataType emailData = new EmailDataType();
           emailData.setEmailingAccepted(false);
           customerData.setEmailData(emailData);

           customerData.setFirstName("John");
```

```
            customerData.setIsCompany(false);
            customerData.setLastName("Doe");
            customerData.setTitleId("MR");

            customerData.setUsualStoreId("302");

            customerData.setBirthDateData(new BirthDateDataType(1, 1, 1970));

            customerData.setCurrencyId("EUR");
            customerData.setLanguageId("FRA");
            customerData.setNationalityId("FRA");

            customerData.setOptinAlternativeEmail(CustomerInformationType.AskCustomer);
            customerData.setOptinEmail(CustomerInformationType.AskCustomer);
            customerData.setOptinMobile(CustomerInformationType.AskCustomer);
            customerData.setOptinOfficePhone(CustomerInformationType.AskCustomer);
            customerData.setOptinPostal(CustomerInformationType.AskCustomer);

            // Create an array of UserDefinedBoolean
            UserDefinedBoolean[] userDefinedBooleans = new UserDefinedBoolean[] {
                            new UserDefinedBoolean(UserDefinedId.value1, true),
                            new UserDefinedBoolean(UserDefinedId.value2, false),
                            new UserDefinedBoolean(UserDefinedId.value3, true)
            };
            customerData.setUserDefinedBooleans(userDefinedBooleans);

            Calendar date = Calendar.getInstance();
            // The month field is an enum, the value is 0-based (0 for January, 1 for February,
etc...)
            // date.set(year, month, day, hour, minute, second)
            date.set(1970, Calendar.JANUARY, 01, 18, 00, 00);
            UserDefinedDate[] userDefinedDates = new UserDefinedDate[] {
                            new UserDefinedDate(UserDefinedId.value1, date)
            };
            customerData.setUserDefinedDates(userDefinedDates);

            UserDefinedText[] userDefinedTexts = new UserDefinedText[] {
                            new UserDefinedText(UserDefinedId.value1, "some text..."),
                            new UserDefinedText(UserDefinedId.value2, "漢字") // non ascii
characters works perfectly
            };
            customerData.setUserDefinedTexts(userDefinedTexts);

            customerData.setSex("M");
            customerData.setShortName("DOE");
            customerData.setVATSystem("FRA");
            customerData.setValidAlternativeEmail(true);
            customerData.setValidEmail(true);
            customerData.setValidMobile(true);
            customerData.setValidOfficePhone(true);

            RetailContext clientContext = new RetailContext("DATABASE");

            // Call
            String response = stub.addNewCustomer(customerData, clientContext);

            // Print response
            System.out.println(response);
    }

}
```

soapclient.js

```
/*****************************************************************************\

Javascript "SOAP Client" library

@version: 2.4 - 2007.12.21
@author: Matteo Casati - http://www.guru4.net/

\*****************************************************************************/

/**
```

```
* Set credentials properties of SOAPClient
* @author: Cegid
*/
function setCredentials(_username, _password, _baseUrl, _authmethod) {
    SOAPClient.username = _username;
    SOAPClient.password = _password;
    SOAPClient.baseurl = _baseUrl;
    SOAPClient.authmethod = _authmethod.toLowerCase();
}

/**
* Call service
* @author: Cegid
*/
function callService(_url, action, _callback, params, _forcedNS, _forcedAction) {

    // Convert Json parameters to Xml
    var xotree = new ObjTree();
    var soapParams = xotree.writeXML(params);

    SOAPClient.invoke(SOAPClient.baseurl + _url, action, soapParams, true, _callback, _forcedNS,
_forcedAction);
}

/*
function SOAPClientParameters()
{
var _pl = new Array();
this.add = function(name, value)
{
_pl[name] = value;
return this;
}
this.toXml = function()
{
var xml = "";
for(var p in _pl)
{
switch(typeof(_pl[p]))
{
case "string":
case "number":
case "boolean":
case "object":
xml += "<" + p + ">" + SOAPClientParameters._serialize(_pl[p]) + "</" + p + ">";
break;
default:
break;
}
}
return xml;
}
}
SOAPClientParameters._serialize = function(o)
{
var s = "";
switch(typeof(o))
{
case "string":
s += o.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;"); break;
case "number":
case "boolean":
s += o.toString(); break;
case "object":
// Date
if(o.constructor.toString().indexOf("function Date()") > -1)
{

var year = o.getFullYear().toString();
var month = (o.getMonth() + 1).toString(); month = (month.length == 1) ? "0" + month : month;
var date = o.getDate().toString(); date = (date.length == 1) ? "0" + date : date;
var hours = o.getHours().toString(); hours = (hours.length == 1) ? "0" + hours : hours;
var minutes = o.getMinutes().toString(); minutes = (minutes.length == 1) ? "0" + minutes :
minutes;
```

```
var seconds = o.getSeconds().toString(); seconds = (seconds.length == 1) ? "0" + seconds :
seconds;
var milliseconds = o.getMilliseconds().toString();
var tzminutes = Math.abs(o.getTimezoneOffset());
var tzhours = 0;
while(tzminutes >= 60)
{
tzhours++;
tzminutes -= 60;
}
tzminutes = (tzminutes.toString().length == 1) ? "0" + tzminutes.toString() :
tzminutes.toString();
tzhours = (tzhours.toString().length == 1) ? "0" + tzhours.toString() : tzhours.toString();
var timezone = ((o.getTimezoneOffset() < 0) ? "+" : "-") + tzhours + ":" + tzminutes;
s += year + "-" + month + "-" + date + "T" + hours + ":" + minutes + ":" + seconds + "." +
milliseconds + timezone;
}
// Array
else if(o.constructor.toString().indexOf("function Array()") > -1)
{
for(var p in o)
{
if(!isNaN(p))   // linear array
{
(/function\s+(\w*)\s*\(/ig).exec(o[p].constructor.toString());
var type = RegExp.$1;
switch(type)
{
case "":
type = typeof(o[p]);
case "String":
type = "string"; break;
case "Number":
type = "int"; break;
case "Boolean":
type = "bool"; break;
case "Date":
type = "DateTime"; break;
}
s += "<" + type + ">" + SOAPClientParameters._serialize(o[p]) + "</" + type + ">"
}
else    // associative array
s += "<" + p + ">" + SOAPClientParameters._serialize(o[p]) + "</" + p + ">"
}
}
// Object or custom function
else
for(var p in o)
s += "<" + p + ">" + SOAPClientParameters._serialize(o[p]) + "</" + p + ">";
break;
default:
break; // throw new Error(500, "SOAPClientParameters: type '" + typeof(o) + "' is not
supported");
}
return s;
}
*/


function SOAPClient() { }

SOAPClient.username = null;
SOAPClient.password = null;
// Added by Cegid
SOAPClient.databaseId = null;
SOAPClient.baseurl = null;
SOAPClient.authmethod = null;

// SOAPClient.invoke = function(url, method, parameters, async, callback)
SOAPClient.invoke = function (url, method, parameters, async, callback, forcedNS, forcedAction) {
    if (async)
        // SOAPClient._loadWsdl(url, method, parameters, async, callback);
        SOAPClient._loadWsdl(url, method, parameters, async, callback, forcedNS, forcedAction);
    else
```

```
        // return SOAPClient._loadWsdl(url, method, parameters, async, callback);
        return SOAPClient._loadWsdl(url, method, parameters, async, callback, forcedNS,
forcedAction);
}

// private: wsdl cache
SOAPClient_cacheWsdl = new Array();

// private: invoke async
// SOAPClient._loadWsdl = function(url, method, parameters, async, callback)
SOAPClient._loadWsdl = function (url, method, parameters, async, callback, forcedNS,
forcedAction) {
    // load from cache?
    var wsdl = SOAPClient_cacheWsdl[url];
    if (wsdl + "" != "" && wsdl + "" != "undefined")
        // return SOAPClient._sendSoapRequest(url, method, parameters, async, callback, wsdl);
        return SOAPClient._sendSoapRequest(url, method, parameters, async, callback, wsdl,
forcedNS, forcedAction);
    // get wsdl
    var xmlHttp = SOAPClient._getXmlHttp();

    /* Conditionnal added by Cegid */
    if (SOAPClient.username && SOAPClient.password) {
        if (SOAPClient.authmethod === "ntlm") {
            var usernamesplit = SOAPClient.username.split("\\");
            Ntlm.setCredentials(usernamesplit[0], usernamesplit[1], SOAPClient.password);
            if (Ntlm.authenticate(url)) {
                xmlHttp.open('GET', url + "?singleWsdl", false);
            }
        }
        else if (SOAPClient.authmethod === "basic") {
            xmlHttp.open("GET", url + "?singleWsdl", async, SOAPClient.username,
SOAPClient.password);
            // Some WS implementations (i.e. BEA WebLogic Server 10.0 JAX-WS) don't support
Challenge/Response HTTP BASIC, so we send authorization headers in the first request
            xmlHttp.setRequestHeader("Authorization", "Basic " +
SOAPClient._toBase64(SOAPClient.username + ":" + SOAPClient.password));
        }
        else
            throw new ReferenceError("Method not valid");
    }
    else
        /* End of conditionnal */
        xmlHttp.open("GET", url + "?singleWsdl", async);



    if (async) {
        xmlHttp.onreadystatechange = function () {
            if (xmlHttp.readyState == 4) {
                // SOAPClient._onLoadWsdl(url, method, parameters, async, callback, xmlHttp);
                /** Conditionnal added by Cegid */
                if (xmlHttp.status == 200) {
                    SOAPClient._onLoadWsdl(url, method, parameters, async, callback, xmlHttp,
forcedNS, forcedAction);
                } else {
                    callback(new Error(xmlHttp.status + ": " + xmlHttp.responseText));
                }
                /** End of conditionnal */
            }
        }
    }
    xmlHttp.send(null);
    if (!async)
        // return SOAPClient._onLoadWsdl(url, method, parameters, async, callback, xmlHttp);
        return SOAPClient._onLoadWsdl(url, method, parameters, async, callback, xmlHttp,
forcedNS, forcedAction);
}
// SOAPClient._onLoadWsdl = function(url, method, parameters, async, callback, req)
SOAPClient._onLoadWsdl = function (url, method, parameters, async, callback, req, forcedNS,
forcedAction) {
    var wsdl = req.responseXML;
    SOAPClient_cacheWsdl[url] = wsdl;      // save a copy in cache
```

```
    return SOAPClient._sendSoapRequest(url, method, parameters, async, callback, wsdl, forcedNS,
forcedAction);
}
// SOAPClient._sendSoapRequest = function(url, method, parameters, async, callback, wsdl)
SOAPClient._sendSoapRequest = function (url, method, parameters, async, callback, wsdl, forcedNS,
forcedAction) {
    // get namespace
    // var ns = (wsdl.documentElement.attributes["targetNamespace"] + "" == "undefined") ?
wsdl.documentElement.attributes.getNamedItem("targetNamespace").nodeValue :
wsdl.documentElement.attributes["targetNamespace"].value;
    var ns = forcedNS == null ? ((wsdl.documentElement.attributes["targetNamespace"] + "" ==
"undefined") ? wsdl.documentElement.attributes.getNamedItem("targetNamespace").nodeValue :
wsdl.documentElement.attributes["targetNamespace"].value) : forcedNS;
    // build SOAP request
    var sr =
        "<?xml version=\"1.0\" encoding=\"utf-8\"?>" +
        "<soap:Envelope " +
        "xmlns:xsi=\"http://www.w3.org/2001/XMLSchema-instance\" " +
        "xmlns:xsd=\"http://www.w3.org/2001/XMLSchema\" " +
        "xmlns:soap=\"http://schemas.xmlsoap.org/soap/envelope/\">" +
        "<soap:Body>" +
        "<" + method + " xmlns=\"" + ns + "\">" +
        // parameters.toXml() +
        parameters +
        "</" + method + "></soap:Body></soap:Envelope>";
    // send request
    var xmlHttp = SOAPClient._getXmlHttp();
    // if (SOAPClient.userName && SOAPClient.password){
    // xmlHttp.open("POST", url, async, SOAPClient.userName, SOAPClient.password);
    if (SOAPClient.username && SOAPClient.password) {
        /* Conditionnal added by Cegid */
        if (SOAPClient.authmethod === "ntlm") {
            var usernamesplit = SOAPClient.username.split("\\");
            Ntlm.setCredentials(usernamesplit[0], usernamesplit[1], SOAPClient.password);
            if (Ntlm.authenticate(url)) {
                xmlHttp.open('POST', url, false);
            }
        }
        else if (SOAPClient.authmethod === "basic") {

            xmlHttp.open("POST", url, async, SOAPClient.username, SOAPClient.password);
            // Some WS implementations (i.e. BEA WebLogic Server 10.0 JAX-WS) don't support
Challenge/Response HTTP BASIC, so we send authorization headers in the first request
            // xmlHttp.setRequestHeader("Authorization", "Basic " +
SOAPClient._toBase64(SOAPClient.userName + ":" + SOAPClient.password));
            xmlHttp.setRequestHeader("Authorization", "Basic " +
SOAPClient._toBase64(SOAPClient.username + ":" + SOAPClient.password));
        }
        else
            throw new ReferenceError("Method not valid");
        /* End of conditionnal */
    }
    else
        xmlHttp.open("POST", url, async);
    // var soapaction = ((ns.lastIndexOf("/") != ns.length - 1) ? ns + "/" : ns) + method;
    var soapaction = forcedAction == null ? (((ns.lastIndexOf("/") != ns.length - 1) ? ns + "/" :
ns) + encodeURIComponent(method)) : forcedAction;
    xmlHttp.setRequestHeader("SOAPAction", soapaction);
    xmlHttp.setRequestHeader("Content-Type", "text/xml; charset=utf-8");
    if (async) {
        xmlHttp.onreadystatechange = function () {
            if (xmlHttp.readyState == 4)
                SOAPClient._onSendSoapRequest(method, async, callback, wsdl, xmlHttp);
        }
    }
    xmlHttp.send(sr);
    if (!async)
        return SOAPClient._onSendSoapRequest(method, async, callback, wsdl, xmlHttp);
}

SOAPClient._onSendSoapRequest = function (method, async, callback, wsdl, req) {
    var o = null;
    var nd = SOAPClient._getElementsByTagName(req.responseXML, method + "Result");
    if (nd.length == 0)
```

```
        nd = SOAPClient._getElementsByTagName(req.responseXML, "return");  // PHP web Service?
    if (nd.length == 0) {
        if (req.responseXML.getElementsByTagName("faultcode").length > 0) {
            if (async || callback)
                // o = new Error(500,
req.responseXML.getElementsByTagName("faultstring")[0].childNodes[0].nodeValue);
                o = new Error("500" + req.responseText,
req.responseXML.getElementsByTagName("faultstring")[0].childNodes[0].nodeValue);
            else
                throw new Error(500,
req.responseXML.getElementsByTagName("faultstring")[0].childNodes[0].nodeValue);
        }
    }
    else
        o = SOAPClient._soapresult2object(nd[0], wsdl);
    if (callback)
        callback(o, req.responseXML);
    if (!async)
        return o;
}
SOAPClient._soapresult2object = function (node, wsdl) {
    var wsdlTypes = SOAPClient._getTypesFromWsdl(wsdl);
    return SOAPClient._node2object(node, wsdlTypes);
}
SOAPClient._node2object = function (node, wsdlTypes) {
    // null node
    if (node == null)
        return null;
    // text node
    if (node.nodeType == 3 || node.nodeType == 4)
        return SOAPClient._extractValue(node, wsdlTypes);
    // leaf node
    if (node.childNodes.length == 1 && (node.childNodes[0].nodeType == 3 ||
node.childNodes[0].nodeType == 4))
        return SOAPClient._node2object(node.childNodes[0], wsdlTypes);
    var isarray = SOAPClient._getTypeFromWsdl(node.nodeName,
wsdlTypes).toLowerCase().indexOf("arrayof") != -1;
    // object node
    if (!isarray) {
        var obj = null;
        if (node.hasChildNodes())
            obj = new Object();
        for (var i = 0; i < node.childNodes.length; i++) {
            var p = SOAPClient._node2object(node.childNodes[i], wsdlTypes);
            obj[node.childNodes[i].nodeName] = p;
        }
        return obj;
    }
    // list node
    else {
        // create node ref
        var l = new Array();
        for (var i = 0; i < node.childNodes.length; i++)
            l[l.length] = SOAPClient._node2object(node.childNodes[i], wsdlTypes);
        return l;
    }
    return null;
}
SOAPClient._extractValue = function (node, wsdlTypes) {
    var value = node.nodeValue;
    switch (SOAPClient._getTypeFromWsdl(node.parentNode.nodeName, wsdlTypes).toLowerCase()) {
        default:
        case "s:string":
            return (value != null) ? value + "" : "";
        case "s:boolean":
            return value + "" == "true";
        case "s:int":
        case "s:long":
            return (value != null) ? parseInt(value + "", 10) : 0;
        case "s:double":
            return (value != null) ? parseFloat(value + "") : 0;
        case "s:datetime":
            if (value == null)
                return null;
```

```
            else {
                value = value + "";
                value = value.substring(0, (value.lastIndexOf(".") == -1 ? value.length :
value.lastIndexOf(".")));
                value = value.replace(/T/gi, " ");
                value = value.replace(/-/gi, "/");
                var d = new Date();
                d.setTime(Date.parse(value));
                return d;
            }
    }
}
SOAPClient._getTypesFromWsdl = function (wsdl) {
    var wsdlTypes = new Array();
    // IE
    var ell = wsdl.getElementsByTagName("s:element");
    var useNamedItem = true;
    // MOZ
    if (ell.length == 0) {
        ell = wsdl.getElementsByTagName("element");
        useNamedItem = false;
    }
    for (var i = 0; i < ell.length; i++) {
        if (useNamedItem) {
            if (ell[i].attributes.getNamedItem("name") != null &&
ell[i].attributes.getNamedItem("type") != null)
                wsdlTypes[ell[i].attributes.getNamedItem("name").nodeValue] =
ell[i].attributes.getNamedItem("type").nodeValue;
        }
        else {
            if (ell[i].attributes["name"] != null && ell[i].attributes["type"] != null)
                wsdlTypes[ell[i].attributes["name"].value] = ell[i].attributes["type"].value;
        }
    }
    return wsdlTypes;
}
SOAPClient._getTypeFromWsdl = function (elementname, wsdlTypes) {
    var type = wsdlTypes[elementname] + "";
    return (type == "undefined") ? "" : type;
}
// private: utils
SOAPClient._getElementsByTagName = function (document, tagName) {
    try {
        // trying to get node omitting any namespaces (latest versions of MSXML.XMLDocument)
        return document.selectNodes(".//*[local-name()=\"" + tagName + "\"]");
    }
    catch (ex) { }
    // old XML parser support
    return document.getElementsByTagName(tagName);
}
// private: xmlhttp factory
SOAPClient._getXmlHttp = function () {
    try {
        if (window.XMLHttpRequest) {
            var req = new XMLHttpRequest();
            // some versions of Moz do not support the readyState property and the onreadystate
event so we patch it!
            if (req.readyState == null) {
                req.readyState = 1;
                req.addEventListener("load",
                    function () {
                        req.readyState = 4;
                        if (typeof req.onreadystatechange == "function")
                            req.onreadystatechange();
                    },
                    false);
            }
            return req;
        }
        if (window.ActiveXObject)
            return new ActiveXObject(SOAPClient._getXmlHttpProgID());
    }
    catch (ex) { }
    throw new Error("Your browser does not support XmlHttp objects");
```

```
}
SOAPClient._getXmlHttpProgID = function () {
    if (SOAPClient._getXmlHttpProgID.progid)
        return SOAPClient._getXmlHttpProgID.progid;
    var progids = ["Msxml2.XMLHTTP.5.0", "Msxml2.XMLHTTP.4.0", "MSXML2.XMLHTTP.3.0",
"MSXML2.XMLHTTP", "Microsoft.XMLHTTP"];
    var o;
    for (var i = 0; i < progids.length; i++) {
        try {
            o = new ActiveXObject(progids[i]);
            return SOAPClient._getXmlHttpProgID.progid = progids[i];
        }
        catch (ex) { };
    }
    throw new Error("Could not find an installed XML parser");
}

SOAPClient._toBase64 = function (input) {
    var keyStr = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/=";
    var output = "";
    var chr1, chr2, chr3;
    var enc1, enc2, enc3, enc4;
    var i = 0;

    do {
        chr1 = input.charCodeAt(i++);
        chr2 = input.charCodeAt(i++);
        chr3 = input.charCodeAt(i++);

        enc1 = chr1 >> 2;
        enc2 = ((chr1 & 3) << 4) | (chr2 >> 4);
        enc3 = ((chr2 & 15) << 2) | (chr3 >> 6);
        enc4 = chr3 & 63;

        if (isNaN(chr2)) {
            enc3 = enc4 = 64;
        } else if (isNaN(chr3)) {
            enc4 = 64;
        }

        output = output + keyStr.charAt(enc1) + keyStr.charAt(enc2) +
            keyStr.charAt(enc3) + keyStr.charAt(enc4);
    } while (i < input.length);

    return output;
}
```

objtree.js

```
/*
 * Based on ObjTree.js from
 * Yusuke Kawasaki http://www.kawa.net/
 *
*/

// If running in Node.js, need package library 'xmldom'.
// Provides the DOMParser object. Everything else stays the same!
if (typeof window === 'undefined') {
    var DOMParser = require('xmldom').DOMParser;

    var window = {};
    window.DOMParser = true;
}

var ObjTree;

// ======================================================================
//  XML.ObjTree -- XML source code from/to JavaScript object like E4X
// ======================================================================

//  constructor

var ObjTree = function () {
```

```
    return this;
};

//  class variables

ObjTree.VERSION = "0.24";

//  object prototype

/** Disabled by Cegid */
// ObjTree.prototype.xmlDecl = '<?xml version="1.0" encoding="UTF-8" ?>\n';
ObjTree.prototype.attr_prefix = '-';
// ObjTree.prototype.overrideMimeType = 'text/xml';
//  method: parseXML( xmlsource )

ObjTree.prototype.parseXML = function (xml) {
    var root;
    if (window.DOMParser) {
        var xmldom = new DOMParser();
        //      xmldom.async = false;           // DOMParser is always sync-mode
        var dom = xmldom.parseFromString(xml, "application/xml");
        if (!dom) return;
        root = dom.documentElement;
    } else if (window.ActiveXObject) {
        xmldom = new ActiveXObject('Microsoft.XMLDOM');
        xmldom.async = false;
        xmldom.loadXML(xml);
        root = xmldom.documentElement;
    }
    if (!root) return;
    return this.parseDOM(root);
};

//  method: parseHTTP( url, options, callback )

ObjTree.prototype.parseHTTP = function (url, options, callback) {
    var myopt = {};
    for (var key in options) {
        myopt[key] = options[key];                  // copy object
    }
    if (!myopt.method) {
        if (typeof (myopt.postBody) == "undefined" &&
            typeof (myopt.postbody) == "undefined" &&
            typeof (myopt.parameters) == "undefined") {
            myopt.method = "get";
        } else {
            myopt.method = "post";
        }
    }
    if (callback) {
        myopt.asynchronous = true;                  // async-mode
        var __this = this;
        var __func = callback;
        var __save = myopt.onComplete;
        myopt.onComplete = function (trans) {
            var tree;
            if (trans && trans.responseXML && trans.responseXML.documentElement) {
                tree = __this.parseDOM(trans.responseXML.documentElement);
            } else if (trans && trans.responseText) {
                tree = __this.parseXML(trans.responseText);
            }
            __func(tree, trans);
            if (__save) __save(trans);
        };
    } else {
        myopt.asynchronous = false;                 // sync-mode
    }
    var trans;
    if (typeof (HTTP) != "undefined" && HTTP.Request) {
        myopt.uri = url;
        var req = new HTTP.Request(myopt);        // JSAN
        if (req) trans = req.transport;
    } else if (typeof (Ajax) != "undefined" && Ajax.Request) {
        var req = new Ajax.Request(url, myopt);   // ptorotype.js
```

```
        if (req) trans = req.transport;
    }
    //  if ( trans && typeof(trans.overrideMimeType) != "undefined" ) {
    //      trans.overrideMimeType( this.overrideMimeType );
    //  }
    if (callback) return trans;
    if (trans && trans.responseXML && trans.responseXML.documentElement) {
        return this.parseDOM(trans.responseXML.documentElement);
    } else if (trans && trans.responseText) {
        return this.parseXML(trans.responseText);
    }
}

//  method: parseDOM( documentroot )

ObjTree.prototype.parseDOM = function (root) {
    if (!root) return;

    this.__force_array = {};
    if (this.force_array) {
        for (var i = 0; i < this.force_array.length; i++) {
            this.__force_array[this.force_array[i]] = 1;
        }
    }

    var json = this.parseElement(root);   // parse root node
    if (this.__force_array[root.nodeName]) {
        json = [json];
    }
    if (root.nodeType != 11) {            // DOCUMENT_FRAGMENT_NODE
        var tmp = {};
        tmp[root.nodeName] = json;          // root nodeName
        json = tmp;
    }

    if (this.soft_arrays) {
        var augmentObject = function (obj) {
            if (typeof obj !== 'string') {
                for (var property in obj) {
                    augmentObject(obj[property]);
                }
                if (!obj.length) {
                    obj.length = 1; obj[0] = obj;
                }
            }
        };
        augmentObject(json);
    }

    return json;
};

//  method: parseElement( element )

ObjTree.prototype.parseElement = function (elem) {
    //  COMMENT_NODE
    if (elem.nodeType == 7) {
        return;
    }

    //  TEXT_NODE CDATA_SECTION_NODE
    if (elem.nodeType == 3 || elem.nodeType == 4) {
        var bool = elem.nodeValue.match(/[^\x00-\x20]/);
        if (bool == null) return;     // ignore white spaces
        return elem.nodeValue;
    }

    var retval;
    var cnt = {};

    //  parse attributes
    if (elem.attributes && elem.attributes.length) {
        retval = {};
        for (var i = 0; i < elem.attributes.length; i++) {
```

```
            var key = elem.attributes[i].nodeName;
            if (typeof (key) != "string") continue;
            var val = elem.attributes[i].nodeValue;
            if (!val) continue;
            key = this.attr_prefix + key;
            if (typeof (cnt[key]) == "undefined") cnt[key] = 0;
            cnt[key]++;
            this.addNode(retval, key, cnt[key], val);
        }
    }

    //  parse child nodes (recursive)
    if (elem.childNodes && elem.childNodes.length) {
        var textonly = true;
        if (retval) textonly = false;        // some attributes exists
        for (var i = 0; i < elem.childNodes.length && textonly; i++) {
            var ntype = elem.childNodes[i].nodeType;
            if (ntype == 3 || ntype == 4) continue;
            textonly = false;
        }
        if (textonly) {
            if (!retval) retval = "";
            for (var i = 0; i < elem.childNodes.length; i++) {
                retval += elem.childNodes[i].nodeValue;
            }
        } else {
            if (!retval) retval = {};
            for (var i = 0; i < elem.childNodes.length; i++) {
                var key = elem.childNodes[i].nodeName;
                if (typeof (key) != "string") continue;
                var val = this.parseElement(elem.childNodes[i]);
                if (!val) continue;
                if (typeof (cnt[key]) == "undefined") cnt[key] = 0;
                cnt[key]++;
                this.addNode(retval, key, cnt[key], val);
            }
        }
    }
    return retval;
};

//  method: addNode( hash, key, count, value )

ObjTree.prototype.addNode = function (hash, key, cnts, val) {
    if (this.__force_array[key]) {
        if (cnts == 1) hash[key] = [];
        hash[key][hash[key].length] = val;      // push
    } else if (cnts == 1) {                   // 1st sibling
        hash[key] = val;
    } else if (cnts == 2) {                   // 2nd sibling
        hash[key] = [hash[key], val];
    } else {                                    // 3rd sibling and more
        hash[key][hash[key].length] = val;
    }
};

//  method: writeXML( tree )

ObjTree.prototype.writeXML = function (tree) {
    var xml = this.hash_to_xml(null, tree);
    /** Edited by Cegid */
    // return this.xmlDecl + xml;
    return xml;
};

//  method: hash_to_xml( tagName, tree )

ObjTree.prototype.hash_to_xml = function (name, tree) {
    var elem = [];
    var attr = [];
    for (var key in tree) {
        if (!tree.hasOwnProperty(key)) continue;
        var val = tree[key];
        if (key.charAt(0) != this.attr_prefix) {
```

```
            if (typeof (val) == "undefined" || val == null) {
                elem[elem.length] = "<" + key + " />";
            } else if (typeof (val) == "object" && val.constructor == Array) {
                elem[elem.length] = this.array_to_xml(key, val);
            } else if (typeof (val) == "object") {
                elem[elem.length] = this.hash_to_xml(key, val);
            } else {
                elem[elem.length] = this.scalar_to_xml(key, val);
            }
        } else {
            attr[attr.length] = " " + (key.substring(1)) + '="' + (this.xml_escape(val)) + '"';
        }
    }
    var jattr = attr.join("");
    var jelem = elem.join("");
    if (typeof (name) == "undefined" || name == null) {
        // no tag
    } else if (elem.length > 0) {
        if (jelem.match(/\n/)) {
            jelem = "<" + name + jattr + ">\n" + jelem + "</" + name + ">\n";
        } else {
            jelem = "<" + name + jattr + ">" + jelem + "</" + name + ">\n";
        }
    } else {
        jelem = "<" + name + jattr + " />\n";
    }
    return jelem;
};

//  method: array_to_xml( tagName, array )

ObjTree.prototype.array_to_xml = function (name, array) {
    var out = [];
    for (var i = 0; i < array.length; i++) {
        var val = array[i];
        if (typeof (val) == "undefined" || val == null) {
            out[out.length] = "<" + name + " />";
        } else if (typeof (val) == "object" && val.constructor == Array) {
            out[out.length] = this.array_to_xml(name, val);
        } else if (typeof (val) == "object") {
            out[out.length] = this.hash_to_xml(name, val);
        } else {
            out[out.length] = this.scalar_to_xml(name, val);
        }
    }
    return out.join("");
};

//  method: scalar_to_xml( tagName, text )

ObjTree.prototype.scalar_to_xml = function (name, text) {
    if (name == "#text") {
        return this.xml_escape(text);
    } else {
        return "<" + name + ">" + this.xml_escape(text) + "</" + name + ">\n";
    }
};

//  method: xml_escape( text )

ObjTree.prototype.xml_escape = function (text) {
    return String(text).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g,
'&gt;').replace(/"/g, '&quot;');
};

if (typeof module !== 'undefined' && module.exports) {
    module.exports = ObjTree;
}

/*
// ======================================================================

=head1 NAME
```

XML.ObjTree -- XML source code from/to JavaScript object like E4X

=head1 SYNOPSIS

```
    var xotree = new XML.ObjTree();
    var tree1 = {
        root: {
            node: "Hello, World!"
        }
    };
    var xml1 = xotree.writeXML( tree1 );        // object tree to XML source
    alert( "xml1: "+xml1 );

    var xml2 = '<?xml version="1.0"?><response><error>0</error></response>';
    var tree2 = xotree.parseXML( xml2 );        // XML source to object tree
    alert( "error: "+tree2.response.error );
```

=head1 DESCRIPTION

XML.ObjTree class is a parser/generater between XML source code
and JavaScript object like E4X, ECMAScript for XML.
This is a JavaScript version of the XML::TreePP module for Perl.
This also works as a wrapper for XMLHTTPRequest and successor to JKL.ParseXML class
when this is used with prototype.js or JSAN's HTTP.Request class.

=head2 JavaScript object tree format

A sample XML source:

```
    <?xml version="1.0" encoding="UTF-8"?>
    <family name="Kawasaki">
        <father>Yasuhisa</father>
        <mother>Chizuko</mother>
        <children>
            <girl>Shiori</girl>
            <boy>Yusuke</boy>
            <boy>Kairi</boy>
        </children>
    </family>
```

Its JavaScript object tree like JSON/E4X:

```
    {
        'family': {
            '-name':    'Kawasaki',
            'father':   'Yasuhisa',
            'mother':   'Chizuko',
            'children': {
                'girl': 'Shiori'
                'boy': [
                    'Yusuke',
                    'Kairi'
                ]
            }
        }
    };
```

Each elements are parsed into objects:

```
    tree.family.father;             # the father's given name.
```

Prefix '-' is inserted before every attributes' name.

```
    tree.family["-name"];           # this family's family name
```

A array is used because this family has two boys.

```
    tree.family.children.boy[0];    # first boy's name
    tree.family.children.boy[1];    # second boy's name
    tree.family.children.girl;      # (girl has no other sisiters)
```

=head1 METHODS

=head2 xotree = new XML.ObjTree()

This constructor method returns a new XML.ObjTree object.

=head2 xotree.force_array = ["rdf:li", "item", "-xmlns"];

This property allows you to specify a list of element names
which should always be forced into an array representation.
The default value is null, it means that context of the elements
will determine to make array or to keep it scalar.

=head2 xotree.attr_prefix = '@';

This property allows you to specify a prefix character which is
inserted before each attribute names.
Instead of default prefix '-', E4X-style prefix '@' is also available.
The default character is '-'.
Or set '@' to access attribute values like E4X, ECMAScript for XML.
The length of attr_prefix must be just one character and not be empty.

=head2 xotree.xmlDecl = '';

This library generates an XML declaration on writing an XML code per default.
This property forces to change or leave it empty.

=head2 tree = xotree.parseXML(xmlsrc);

This method loads an XML document using the supplied string
and returns its JavaScript object converted.

=head2 tree = xotree.parseDOM(domnode);

This method parses a DOM tree (ex. responseXML.documentElement)
and returns its JavaScript object converted.

=head2 tree = xotree.parseHTTP(url, options);

This method loads a XML file from remote web server
and returns its JavaScript object converted.
XMLHTTPRequest's synchronous mode is always used.
This mode blocks the process until the response is completed.

First argument is a XML file's URL
which must exist in the same domain as parent HTML file's.
Cross-domain loading is not available for security reasons.

Second argument is options' object which can contains some parameters:
method, postBody, parameters, onLoading, etc.

This method requires JSAN's L<HTTP.Request> class or prototype.js's Ajax.Request class.

=head2 xotree.parseHTTP(url, options, callback);

If a callback function is set as third argument,
XMLHTTPRequest's asynchronous mode is used.

This mode calls a callback function with XML file's JavaScript object converted
after the response is completed.

=head2 xmlsrc = xotree.writeXML(tree);

This method parses a JavaScript object tree
and returns its XML source generated.

=head1 EXAMPLES

=head2 Text node and attributes

If a element has both of a text node and attributes
or both of a text node and other child nodes,
text node's value is moved to a special node named "#text".

```
    var xotree = new XML.ObjTree();
    var xmlsrc = '<span class="author">Kawasaki Yusuke</span>';
    var tree = xotree.parseXML( xmlsrc );
```

```
    var class = tree.span["-class"];        # attribute
    var name  = tree.span["#text"];         # text node
```

=head2 parseHTTP() method with HTTP-GET and sync-mode

HTTP/Request.js or prototype.js must be loaded before calling this method.

```
    var xotree = new XML.ObjTree();
    var url = "http://example.com/index.html";
    var tree = xotree.parseHTTP( url );
    xotree.attr_prefix = '@';                   // E4X-style
    alert( tree.html["@lang"] );
```

This code shows C<lang=""> attribute from a X-HTML source code.

=head2 parseHTTP() method with HTTP-POST and async-mode

Third argument is a callback function which is called on onComplete.

```
    var xotree = new XML.ObjTree();
    var url = "http://example.com/mt-tb.cgi";
    var opts = {
        postBody:   "title=...&excerpt=...&url=...&blog_name=..."
    };
    var func = function ( tree ) {
        alert( tree.response.error );
    };
    xotree.parseHTTP( url, opts, func );
```

This code send a trackback ping and shows its response code.

=head2 Simple RSS reader

This is a RSS reader which loads RDF file and displays all items.

```
    var xotree = new XML.ObjTree();
    xotree.force_array = [ "rdf:li", "item" ];
    var url = "http://example.com/news-rdf.xml";
    var func = function( tree ) {
        var elem = document.getElementById("rss_here");
        for( var i=0; i<tree["rdf:RDF"].item.length; i++ ) {
            var divtag = document.createElement( "div" );
            var atag = document.createElement( "a" );
            atag.href = tree["rdf:RDF"].item[i].link;
            var title = tree["rdf:RDF"].item[i].title;
            var tnode = document.createTextNode( title );
            atag.appendChild( tnode );
            divtag.appendChild( atag );
            elem.appendChild( divtag );
        }
    };
    xotree.parseHTTP( url, {}, func );
```

=head2 XML-RPC using writeXML, prototype.js and parseDOM

If you wish to use prototype.js's Ajax.Request class by yourself:

```
    var xotree = new XML.ObjTree();
    var reqtree = {
        methodCall: {
            methodName: "weblogUpdates.ping",
            params: {
                param: [
                    { value: "Kawa.net xp top page" },  // 1st param
                    { value: "http://www.kawa.net/" }   // 2nd param
                ]
            }
        }
    };
    var reqxml = xotree.writeXML( reqtree );       // JS-Object to XML code
    var url = "http://example.com/xmlrpc";
    var func = function( req ) {
        var resdom = req.responseXML.documentElement;
        xotree.force_array = [ "member" ];
```

```
        var restree = xotree.parseDOM( resdom );   // XML-DOM to JS-Object
        alert( restree.methodResponse.params.param.value.struct.member[0].value.string );
    };
    var opt = {
        method:         "post",
        postBody:       reqxml,
        asynchronous:   true,
        onComplete:     func
    };
    new Ajax.Request( url, opt );

=head1 AUTHOR

Yusuke Kawasaki http://www.kawa.net/

=head1 COPYRIGHT AND LICENSE

Copyright (c) 2005-2006 Yusuke Kawasaki. All rights reserved.
This program is free software; you can redistribute it and/or
modify it under the Artistic license. Or whatever license I choose,
which I will do instead of keeping this documentation like it is.

=cut
// ======================================================================
*/
```

CookieSoap.php

```php
<?php
  $filename = "cookie";

  function WriteCookie($cookie)
  {
   global $filename;
   $fp = fopen($filename, "w");
   fputs($fp, $cookie);
   fclose($fp);
  }

  function ReadCookie()
  {
   global $filename;
   $fp = fopen($filename, "r");
   $cookie = fgets($fp);
   return $cookie;
  }
?>
```

HelloWorld_cookie.php

```php
<?php
  include('CookieSoap.php');
  $url = "http://localhost/CBR_12.1/WorkerProcessService.svc?singleWsdl";
  // client proxy
  $client = new SoapClient($url);

  // request complex type
  $request = new StdClass();
  $request->text = "test";
  $request->clientContext = new StdClass();
  $request->clientContext->DatabaseId = "DATABASE";

  // retrieve the last cookie stored
  $client->__setCookie("CbrAuth", ReadCookie());

  // call
  try {
   $resu = $client->HelloWorld($request);
  } catch (Exception $e) {
   if ($e->getMessage() == "Unauthorized") // if 401 is returned...
   {
           // ...create the client with authentication
           $client = new SoapClient($url,
```

```
                    array(
                            "login" => "DOMAIN\\USERNAME",
                            "password" => "PASSWORD"
                    )
            );
    }
    // call
    $resu = $client->HelloWorld($request);
    // store the cookie
    WriteCookie($client->_cookies["CbrAuth"][0]);
   }

   print_r($resu);
?>
```